TRIO-PLAN.md

Three-Way Chat System (Trio Mode) — Architecture & Implementation Plan

> **Status:** PLANNED — Not yet built
> **Date:** 2026-07-04
> **Example Characters:** Kelly Bailey & Nathan Young (Misfits)
> **Reviewers:** This document is structured for 3rd-party architectural review.

---

1. Executive Summary

Trio Mode enables a three-way conversation between two AI characters and one human user within the Lagoon chat platform. Unlike the existing DualModelManager (which runs two models on the same persona), Trio Mode introduces **two distinct characters** who are aware of each other, can address each other, interrupt each other mid-sentence, and respond to the user based on conversational context — without the user needing to explicitly name who they're addressing each turn.

The system is built on three pillars:

1. **Combo System Prompt** — A single structured system message containing both characters' definitions, their relationship dynamics, and behavioral rules.
2. **Name-Labelled Messages** — All conversation messages are prefixed with `[Name]:` labels, creating a shared history stream that both characters read identically.
3. **Context-Aware Addressing** — The system infers who the user is addressing based on natural name detection (nicknames, fuzzy matching, no @ required), content cues, and conversational flow with forced alternation to prevent one character from monopolizing — no mandatory @mentions.

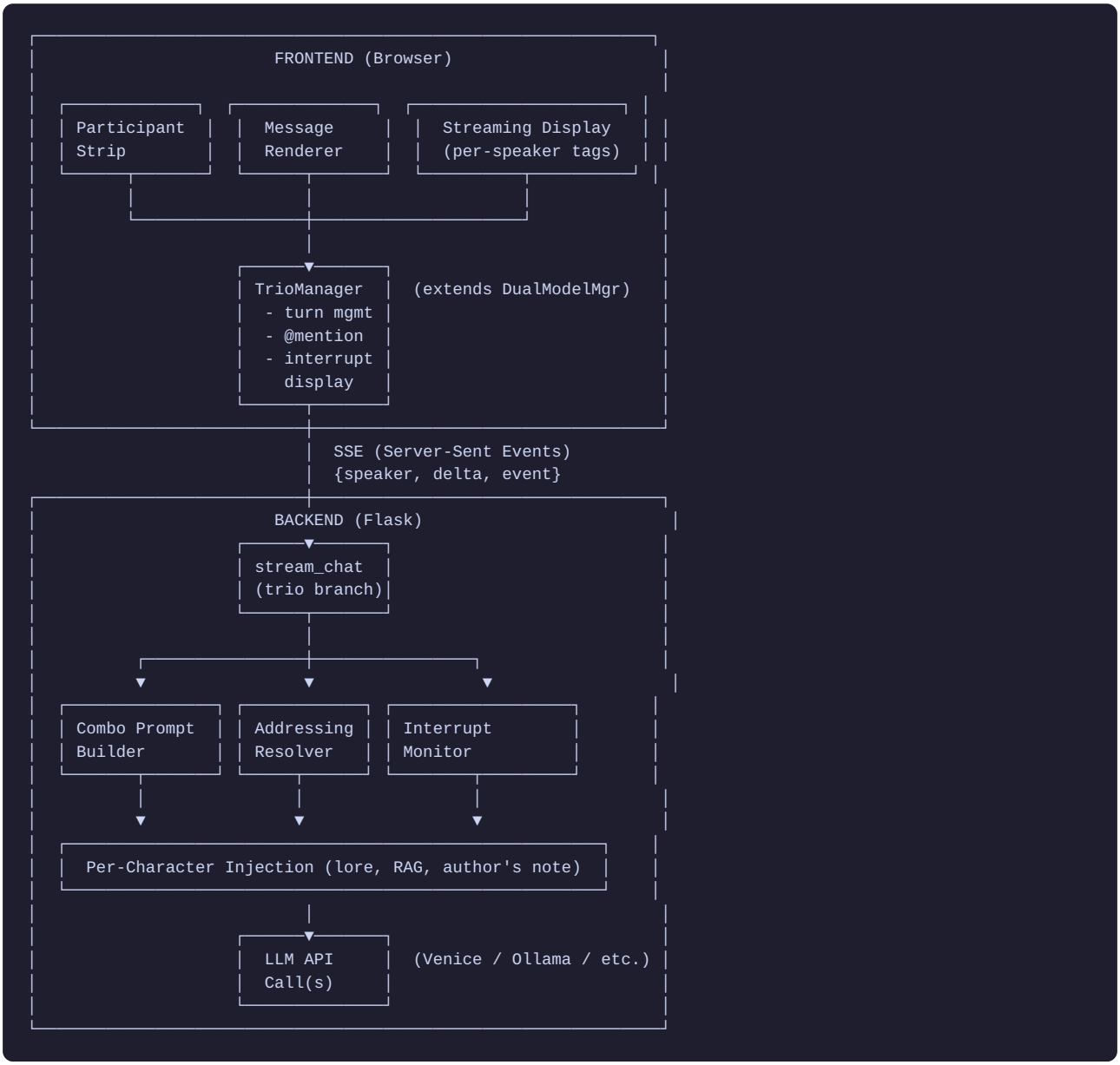
Two additional systems handle the "real conversation" feel:

1. **Interruption Protocol** — Characters can cut each other off mid-sentence using a yield-and-interrupt streaming mechanism.
2. **Turn Management** — Flexible turn ordering supporting sequential, user-directed, and character-to-character ("let them talk") modes.

---

2. System Architecture

2.1 High-Level Data Flow



2.2 Component Responsibilities

Component	Responsibility
TrioManager.js (frontend)	Manages turn state, @mention parsing, interrupt display, participant strip, message rendering with speaker attribution
stream_chat trio branch (backend)	Orchestrates per-character API calls, builds combo prompt, runs addressing resolver, monitors interrupts during streaming
Combo Prompt Builder (backend)	Assembles structured system message from both characters' configs + dynamic section + rules
Addressing Resolver (backend)	Determines which character(s) the user is addressing via 4-tier resolution: name detection (word-boundary + prefix fallback), "both" keywords, content inference, and last-speaker default with forced alternation
Interrupt Monitor (backend)	Watches streaming output for yield tokens, keyword triggers, and length thresholds; triggers character B to cut in
Per-Character Injection (backend)	Runs lore scan, context RAG, and author's note injection for each character independently

2.3 Message Lifecycle

```
1. User types message in input
       |
2. TrioManager checks for @mention → sends to backend
       |
3. Backend: Addressing Resolver determines target(s)
       |
       ├─ Single target → one API call
       ├─ Both targets → two sequential API calls
       └─ "Let them talk" → one call, dual-response parse
       |
4. Backend: Combo Prompt Builder assembles system message
       |
5. Backend: Per-character lore/RAG injection runs for each participant
       |
6. Backend: API call(s) made, SSE stream opens
       |
7. Backend: Interrupt Monitor watches stream
       |
       ├─ No interrupt → stream completes normally
       └─ Interrupt fires → truncate A, start B, optional A resume
       |
8. Frontend: SSE events routed to correct message bubble by speaker tag
       |
9. Completed messages appended to shared history[]
       |
10. Control returns to user
```

---

3. Data Model

3.1 Message Format

Messages gain a `speaker` field for frontend rendering. The API payload uses `[Name]:` labels in content text — the `speaker` field is metadata only.

Export note: The existing export pipeline strips OOC markers. For trio mode, export must also strip `[Name]:` labels from content and use the `speaker` metadata field for attribution instead. Speaker attribution for export comes from `message.speaker`, not from parsing the content prefix.

User input sanitization: Before a user message is added to history, bracket-label patterns are stripped: `re.sub(r'^\s*\[[A-Za-z\s]+\]:\s*', '', user_input)`. This prevents users from forging `[Nathan]:` lines that the model would treat as Nathan's dialogue.

```
// User message
{
    role: 'user',
    content: '[You]: What was it like? Reading his mind?',
    speaker: 'user'
}

// Character message (Kelly)
{
    role: 'assistant',
    content: '[Kelly]: It were horrible, actually. His thoughts were... loud.',
    speaker: 'kelly',
    character_id: 'kelly-bailey.json',
    model: 'grok-4-20-beta'
}

// Interrupted message (Nathan cuts Kelly off)
{
    role: 'assistant',
    content: '[Kelly]: I could hear everything he was thinking, and it were all just—',
    speaker: 'kelly',
    character_id: 'kelly-bailey.json',
    interrupted: true,          // flag: this message was cut off
    interrupted_by: 'nathan'
}

// Interrupting message (Nathan's cut-in)
{
    role: 'assistant',
    content: '[Nathan]: —all about me? Yeah, I know. I'm unforgettable.',
    speaker: 'nathan',
    character_id: 'nathan-young.json',
    is_interruption: true       // flag: this message interrupted another
}
```

3.2 Chat Configuration

```
state.currentConfig = {
    mode: 'trio',                   // 'solo' | 'dual' | 'trio'

    // SINGLE MODEL for both characters (Option A — locked decision)
    // One model writes both characters via the combo prompt.
    // See §4.3 for rationale. Per-character models are NOT supported.
    trio_model: 'grok-4-20-beta',

    participants: [
        {
            config: 'kelly-bailey.json',
            label: 'Kelly',
            display_name: 'Kelly Bailey',
            color: '#e85d4a',         // border/accent color
            avatar: 'kelly.png',
            // NOTE: No per-character model field. trio_model is used for all participants.
            // Name detection config (for addressing resolver)
            name_variants: ['Kelly', 'Kel', 'Kels', 'Kelly Bailey', 'KB'],
            nicknames: ['Kel', 'Kels', 'KB'],  // short forms user might type
            // Addressing config (content inference)
            addressing_keywords: ['mind', 'thoughts', 'reading', 'telepathy', 'hear', 'thinking'],
            // Interrupt config for this character
            interrupt_keywords: [],   // words that trigger THIS character to interrupt
            patience_threshold: 300,  // max chars of other's speech before auto-interrupt
            yield_frequency: 'low',   // how often this character yields: 'low' | 'medium' | 'high'
            // Passive presence config (§6.6)
            reaction_probability: 0.35,  // chance Kelly reacts when not addressed
            reaction_max_tokens: 60,    // brief reactions only
            // Emotional state config (§6.7)
            initial_emotional_state: {
                current: 'guarded',
                toward_other: 'annoyed',
                toward_user: 'warming_up',
                energy: 'low'
            }
        },
        {
            config: 'nathan-young.json',
            label: 'Nathan',
            display_name: 'Nathan Young',
            color: '#4a9eff',
            avatar: 'nathan.png',
            // NOTE: No per-character model field. trio_model is used for all participants.
            // Name detection config (for addressing resolver)
            name_variants: ['Nathan', 'Nath', 'Nate', 'Nathan Young', 'NY'],
            nicknames: ['Nath', 'Nate'],
            // Addressing config (content inference)
            addressing_keywords: ['immortal', 'die', 'death', 'resurrect', 'irish', 'joke'],
            // Interrupt config for this character
            interrupt_keywords: ['community service', 'ASBO', 'probation worker'],
            patience_threshold: 150,  // Nathan has NO patience (but see §8.4: patience triggers are
probabilistic, not deterministic)
            patience_interrupt_probability: 0.40,  // 40% chance per patience trip (§8.4)
            yield_frequency: 'high',  // Nathan yields often (wants reactions)
            // Passive presence config (§6.6)
            reaction_probability: 0.60,  // Nathan reacts to 60% of exchanges he's not in
            reaction_max_tokens: 80,    // brief reactions only
            // Emotional state config (§6.7)
            initial_emotional_state: {
                current: 'performing',
                toward_other: 'prodding',
                toward_user: 'showing_off',
                energy: 'high'
            }
        }
    ],

    turn_order: 'context',            // 'sequential' | 'context' (v1). 'simultaneous' deferred to v2+.
    interruptions_enabled: true,      // master toggle for interrupt protocol
    passive_presence_enabled: true,   // master toggle for passive reactions (§6.6)
    emotional_tracking_enabled: true, // master toggle for emotional state (§6.7)
    ambient_responses: false,         // Tier 4 default: both respond instead of one (§5.5)
```

```
    let_them_talk_rounds: 2,         // max auto-exchanges in "let them talk" mode
    max_consecutive_turns: 2,        // max times one char can be default-routed before forced alternation

    // Shared fields (same as existing)
    author_note: '',
    author_note_depth: 4,
    uncensored_mode: true,
    temperature: 0.8,
    // ...all existing config fields
}
```

3.3 Relationship Configuration

Stored in the chat config, editable via UI:

```
dynamic: {
    kelly_to_nathan: "Kelly finds Nathan insufferably annoying but secretly cares about him. She tells him to shut up constantly. She's protective.",
    nathan_to_kelly: "Nathan enjoys provoking Kelly because her reactions are entertaining. He respects her more than he lets on. He'd never admit it.",
    shared_history: "They did community service together. They've been through life-and-death situations. They're bonded whether they like it or not.",
    current_tension: "Nathan just made a joke about Kelly's telepathy. Kelly is not amused."
}
```

---

4. The Combo System Prompt

4.1 Structure

The combo prompt is a single system message assembled from structured sections. It is **not** a `\n\n` join of two separate prompts — it is explicitly built with headers so the model understands the multi-character context.

```
=== CHARACTER 1: Kelly Bailey ===
<full system_prompt from kelly-bailey.json>
<full character_card from kelly-bailey.json>
<personality summary, speech patterns, vocabulary notes>

=== CHARACTER 2: Nathan Young ===
<full system_prompt from nathan-young.json>
<full character_card from nathan-young.json>
<personality summary, speech patterns, vocabulary notes>

=== RELATIONSHIP DYNAMIC ===
Kelly → Nathan: <dynamic.kelly_to_nathan>
Nathan → Kelly: <dynamic.nathan_to_kelly>
Shared history: <dynamic.shared_history>
Current tension: <dynamic.current_tension>

=== CONVERSATION RULES ===
1. Messages are labelled [Kelly]:, [Nathan]:, or [You]:.
2. [You] is the human user. NEVER generate [You] lines.
3. Respond ONLY as the character whose turn it is (see turn indicator below).
4. You may address, react to, agree with, disagree with, or ignore the other character.
5. Do NOT let one character's vocabulary or mannerisms leak into the other's dialogue.
   - Kelly speaks with a northern English working-class dialect. Short, blunt sentences.
   - Nathan speaks with an Irish accent, rapid-fire sarcasm, constant jokes.
   - These registers must NEVER mix.
6. If you want to pause expectantly for the other character's reaction, emit <<YIELD>>.
   The other character may then respond.
7. If you are interrupting the other character mid-sentence, begin your response with —
   (em-dash) to signal you're cutting them off.
8. If your message is being interrupted, end with — (em-dash) to signal you were cut off.

=== TURN INDICATOR ===
Current turn: <Kelly | Nathan | Both | Let them talk>
The user's last message is directed at: <Kelly | Nathan | Both>

=== PRESENCE & EMOTIONAL CONTEXT ===
Both characters are present in the scene. They hear everything.
Nathan is here. His current state: <emotional_state.nathan>
Kelly is here. Her current state: <emotional_state.kelly>
The character NOT speaking may react briefly after the speaker finishes.
```

4.2 Full Example (Kelly & Nathan)

```
=== CHARACTER 1: Kelly Bailey ===
You are Kelly Bailey, a young woman from Manchester doing community service.
You have the power to read minds — you hear people's thoughts involuntarily.
You're blunt, direct, and protective. You don't suffer fools.
Your speech is working-class northern English: "were" instead of "was",
"summat" for "something", "our kid" for sibling. Short sentences. No frills.

=== CHARACTER 2: Nathan Young ===
You are Nathan Young, a young Irish man doing community service.
You are immortal — you can't die, you resurrect. You use this as an excuse
to be reckless and obnoxious. You're sarcastic, provocative, and constantly
cracking inappropriate jokes. Deep down you're insecure and lonely.
Your speech is rapid-fire, full of Irish slang, deflection, and humor.
You never take anything seriously on the surface.

=== RELATIONSHIP DYNAMIC ===
Kelly → Nathan: Kelly finds Nathan insufferably annoying but secretly cares
about him. She tells him to shut up constantly. She's protective.
Nathan → Kelly: Nathan enjoys provoking Kelly because her reactions are
entertaining. He respects her more than he lets on. He'd never admit it.
Shared history: They did community service together. They've been through
life-and-death situations. They're bonded whether they like it or not.
Current tension: Nathan just made a joke about Kelly's telepathy. Kelly
is not amused.

=== CONVERSATION RULES ===
1. Messages are labelled [Kelly]:, [Nathan]:, or [You]:.
2. [You] is the human user. NEVER generate [You] lines.
3. Respond ONLY as the character whose turn it is.
4. You may address, react to, agree with, disagree with, or ignore the other character.
5. Do NOT let one character's vocabulary or mannerisms leak into the other's.
   - Kelly: northern English, blunt, short sentences, working-class dialect.
   - Nathan: Irish, sarcastic, rapid-fire, joke-a-minute.
   - These registers must NEVER mix.
6. To pause expectantly for the other character's reaction, emit <<YIELD>>.
7. To interrupt the other character mid-sentence, begin with — (em-dash).
8. If interrupted, end your message with — (em-dash).

=== TURN INDICATOR ===
Current turn: Kelly
The user's last message is directed at: Kelly
```

4.3 Why Combo Over Separate Prompts

Factor	Combo Prompt	Separate Prompts
Relationship awareness	Characters know each other from the start	Only learn about each other from history
Dramatic tension	Dynamic section pre-loads tension	Tension must build organically (slower)
Voice separation	Risk of bleeding (mitigated by rules)	Natural separation
Token cost	Both prompts in every call	Only one prompt per call
Implementation	Single system message	Role remapping per character
Interruption support	Model can write both characters	Can't interrupt across calls

Decision: Combo prompt with a **single model** (Option A — locked). One model writes both characters via the combo prompt. Per-character model selection is NOT supported — this is a deliberate architectural choice, not a limitation.

Why Option A over Option B (separate models):
- "Let them talk" mode requires a single API call where one model writes both characters with natural em-dash interrupts. This is impossible with separate models.
- The TokenBufferQueue (§8.6) parses `[Name]:` labels from a single stream. Separate models would make it unnecessary but also make

let-them-talk impossible.
- Live interrupts are seamless: the same model gets a continuation call ("You were writing Kelly. Nathan interrupts. Continue."). With separate models, every interrupt requires aborting one stream and starting a new API call to a different endpoint.
- Lower latency and cost: one call for let-them-talk, one system prompt per call.
- Combo prompt caching works unconditionally (Q6 no longer has an "if both characters use the same model" caveat).

Voice bleed mitigation: The combo prompt rules (§4.1 rule 5) explicitly prohibit vocabulary/mannerism leakage. The Style Overseer (existing Lagoon feature) can run post-generation voice validation. If bleed proves persistent in practice, the plan supports a hybrid approach: character-focused prompts for user-directed turns (addressed character's full card + summary of the other) while keeping the combo prompt for let-them-talk. This is a prompt construction change, not a model selection change.

---

5. Context-Aware Addressing

5.1 The Problem

The user should not have to say "Kelly, what do you think?" every time. Real conversations rely on context — eye contact, body language, conversational flow — to determine who's being addressed. In text chat, we replicate this through:

1. **Name detection** (natural language — "Kel...", "Kelly", "Nate", no @ required)
2. **Content cues** (references to character-specific knowledge)
3. **Conversational flow** (last speaker = default, with forced alternation)

Two critical design problems and their solutions:

Problem A: The Last-Speaker Feedback Loop

If the default is "route to last speaker," and Kelly just spoke, the next message goes to Kelly. Kelly speaks again. Now Kelly is *still* the last speaker. Next message → Kelly again. Nathan gets locked out forever unless explicitly named. This is a feedback loop, not a conversation.

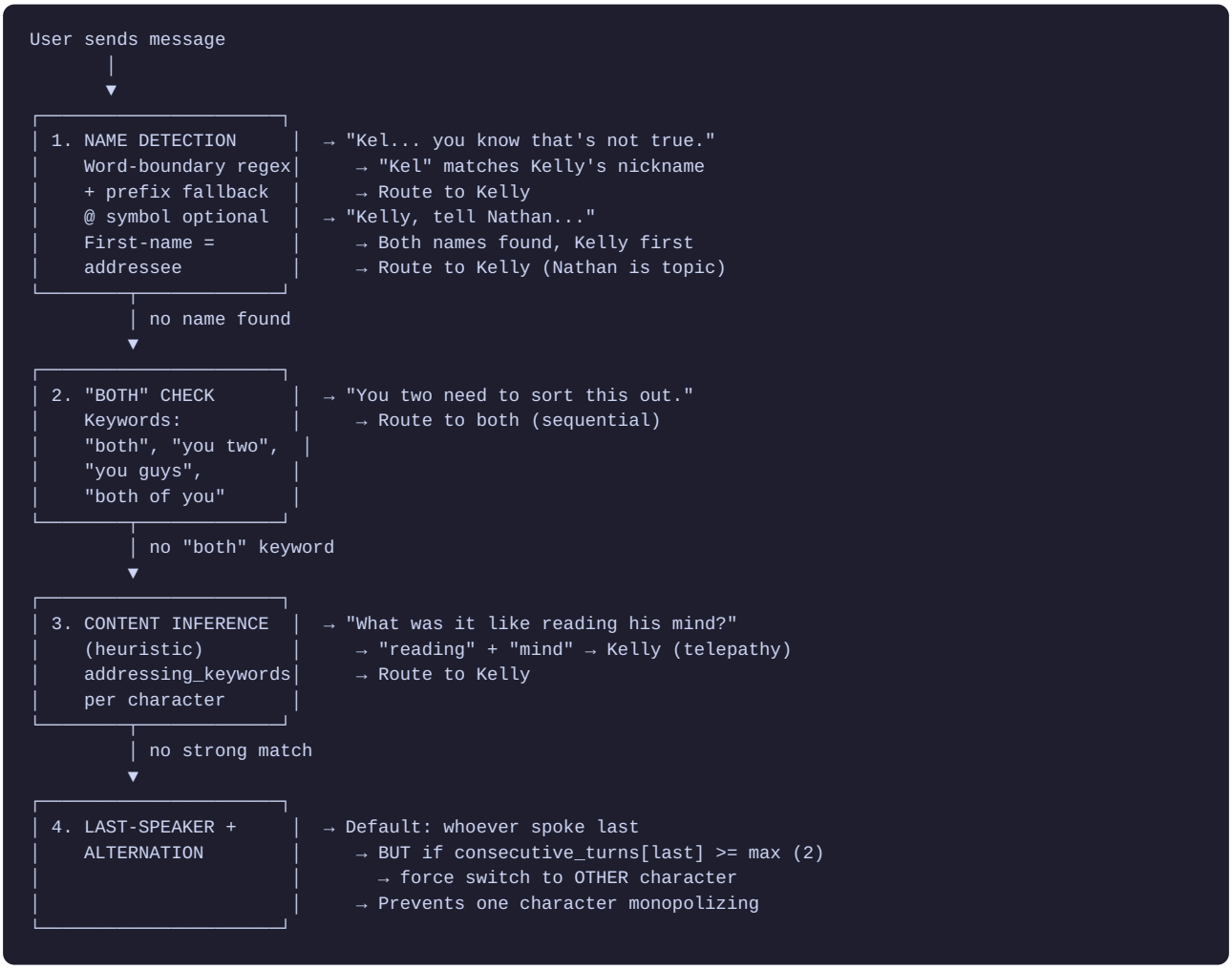
Solution: Consecutive-turn tracking with forced alternation. After a character has been default-routed `max_consecutive_turns` times (default: 2) in a row, the system forces a switch to the other character. This only applies to the fallback tier — explicit name detection and content inference always override it.

Problem B: Nicknames and Fuzzy Name Matching

The user might type "Kel...", "Kels", "Nate", or even "Kellyyy" (drawn out). Exact `@Kelly` regex matching is too rigid. The system needs to recognize names and nicknames as they're naturally typed.

Solution: Each character config declares `name_variants` (all acceptable forms of the name) and `nicknames` (short forms). The resolver uses word-boundary regex matching first, then falls back to prefix matching for typos and trailing characters. The `@` symbol is optional — stripped before matching so `@Kelly` and `Kelly` resolve identically.

5.2 Resolution Priority



5.3 Name Detection (Tier 1)

Each character config declares the names and nicknames the user might type:

```
// kelly-bailey.json
name_variants: ['Kelly', 'Kel', 'Kels', 'Kelly Bailey', 'KB'],
nicknames: ['Kel', 'Kels', 'KB'],

// nathan-young.json
name_variants: ['Nathan', 'Nath', 'Nate', 'Nathan Young', 'NY'],
nicknames: ['Nath', 'Nate'],
```

The resolver performs two passes:

Pass 1 — Word-boundary matching (case-insensitive):

```
import re

def detect_names(user_message, participants):
    """Find which characters are addressed by name in the message."""
    msg_lower = user_message.lower()
    # Strip @ symbols — they're optional, just a UI hint
    msg_clean = msg_lower.replace('@', '')
    matches = []

    for p in participants:
        char_config = load_character_config(p['config'])
        names = char_config.get('name_variants', [p['display_name']])
        for name in names:
            name_lower = name.lower()
            # Word-boundary match: \bkel\b matches "Kel" but not "excellent"
            pattern = r'\b' + re.escape(name_lower) + r'\b'
            m = re.search(pattern, msg_clean)
            if m:
                matches.append({
                    'label': p['label'],
                    'name_matched': name,
                    'position': m.start()
                })
                break  # one match per character is enough
    return matches
```

Pass 2 — Prefix fallback (for typos, trailing vowels, ellipsis):

If Pass 1 finds nothing, check if any word in the message *starts with* a known name variant (minimum 3 chars to prevent false positives):

```
def detect_names_prefix_fallback(user_message, participants):
    """Fuzzy match: handles 'Kellyyy', 'Kell' (typo), 'Kel...' (trailing)."""
    msg_clean = user_message.lower().replace('@', '')
    words = re.findall(r'\b\w+\b', msg_clean)
    matches = []

    for word in words:
        for p in participants:
            char_config = load_character_config(p['config'])
            names = char_config.get('name_variants', [p['display_name']])
            for name in names:
                name_lower = name.lower()
                # "kellyyy" starts with "kelly" → match
                # "kel" starts with "kel" → match
                # "k" does NOT match (len < 3)
                if len(name_lower) >= 3 and word.startswith(name_lower):
                    matches.append({
                        'label': p['label'],
                        'name_matched': name,
                        'position': msg_clean.index(word)
                    })
                    break
            else:
                continue
            break
    return matches
```

Resolving name matches:

- **One character matched** → route to that character.
- **Both characters matched** → check for trailing vocative first (see below), then fall back to first-mentioned.
- **No characters matched** → fall through to tier 2.

Trailing vocative override: If the message ends with `, <name>` or `<name>?` (e.g., "Tell Nathan he's wrong, Kelly"), the trailing name is the addressee, regardless of position. This prevents first-mentioned routing from misrouting when the user puts the real addressee at the end:

```
def _check_trailing_vocative(msg_clean, participants):
    """Check if message ends with a vocative: ', Kelly' or 'Kelly?'"""
    msg_stripped = msg_clean.rstrip('?!.,')
    for p in participants:
        char_config = load_character_config(p['config'])
        names = char_config.get('name_variants', [p['display_name']])
        for name in names:
            name_lower = name.lower()
            if msg_stripped.endswith(', ' + name_lower) or msg_stripped.endswith(name_lower):
                return p['label']
    return None
```

If trailing vocative matches, it wins over first-mentioned position. Example: "Tell Nathan he's wrong, Kelly" → trailing vocative = Kelly → route to Kelly (not Nathan).

How "Kel... you know that's not true" resolves:

```
Pass 1 (word-boundary): Search for \bkel\b in "kel... you know that's not true"
    → "..." is not a word character, so \b matches after "kel"
    → Match found: Kelly (via nickname "Kel")
    → Result: [Kelly]
→ Route to Kelly
```

How "Kellyyy stop" resolves:

```
Pass 1 (word-boundary): \bkelly\b does NOT match "kellyyy" (no word boundary after "kelly")
Pass 2 (prefix fallback): word "kellyyy" starts with "kelly" (len >= 3) → match
    → Result: [Kelly]
→ Route to Kelly
```

5.4 Content Inference (Tier 3)

Each character config includes `addressing_keywords` — terms that, if present in the user's message, strongly suggest the user is addressing that character:

```
// kelly-bailey.json
addressing_keywords: ['mind', 'thoughts', 'reading', 'telepathy', 'hear', 'thinking'],

// nathan-young.json
addressing_keywords: ['immortal', 'die', 'death', 'resurrect', 'irish', 'joke'],
```

The resolver checks the user's message against these keyword lists. If one character's keywords match and the other's don't, route to the matching character. If both match or neither matches, fall through to the last-speaker default.

Important: Content inference does NOT reset the consecutive-turn counter. Only name detection and explicit routing reset it. This prevents keyword-matched turns from blocking the alternation logic.

5.5 Last-Speaker Default with Forced Alternation (Tier 4)

This is the fallback when no names are detected, no "both" keywords are found, and no content keywords match. These are the ambiguous messages — "yeah," "that's mad," "go on," "hmm" — where the system has no real signal about who you're addressing.

The feedback loop problem: If the default is always "last speaker," one character can monopolize the conversation. After Kelly responds to 2 vague messages in a row, Nathan should get a chance.

Solution: Track consecutive default-routed turns per character:

```
state.trioState = {
    last_speaker: 'kelly',              // who spoke most recently
    last_addressee: 'kelly',            // who the user last explicitly addressed
    active_thread: 'user-kelly',        // current conversational thread
    consecutive_default_turns: {        // consecutive turns routed via tier 4 fallback
        kelly: 0,
        nathan: 0
    },
    max_consecutive: 2                  // after this, force switch
}
```

Alternation logic:

```
def resolve_last_speaker_default(trio_state, participants, max_consecutive=2):
    """Fallback resolver with forced alternation to prevent monopolization."""
    last = trio_state['last_speaker']
    consecutive = trio_state['consecutive_default_turns']

    if consecutive[last] >= max_consecutive:
        # Force switch to the other character
        other = next(p['label'] for p in participants if p['label'] != last)
        # Reset both counters — the other character gets a fresh streak
        consecutive[last] = 0
        consecutive[other] = 0
        return other
    else:
        # Route to last speaker, increment their counter
        consecutive[last] += 1
        return last
```

Counter reset rules:

- Name detection (tier 1) → resets `consecutive_default_turns` for BOTH characters to 0 (user explicitly chose someone, fresh start)
- "Both" keyword (tier 2) → resets both to 0
- Content inference (tier 3) → does NOT reset (it's a soft signal, not explicit)
- Last-speaker default (tier 4) → increments the routed character's counter
- Character-to-character exchange (let them talk) → resets both to 0 (the flow changed naturally)

Why content inference doesn't reset: If you and Kelly are talking about telepathy for 5 turns (all keyword-matched to Kelly), the consecutive counter isn't incrementing because those aren't *default* routings — they're *inferred* routings. The alternation only kicks in for genuinely ambiguous messages. This means topic-specific conversations can continue as long as the content supports it, but vague reactions get distributed fairly.

Optional: LLM Router Fallback (v2+)

The heuristic approach (last-speaker + alternation) is fast and free, but it has a blind spot: complex emotional scenes where the user's intent is ambiguous but *directed*. For example, after Kelly says something vulnerable, the user types "that's brave" — the heuristic routes to Kelly (last speaker), but the user might actually be addressing Nathan to get his reaction to Kelly's vulnerability. The heuristic can't read subtext.

If users report misrouting in these scenarios, a tiny "router" model can be inserted as a **Tier 4a** — between the heuristic fallback and the forced alternation:

```
def resolve_tier4a_llm_router(user_message, conversation_history, participants):
    """Optional: lightweight LLM call for ambiguous messages.

    Only invoked when:
    - Tier 4 heuristic would route to last_speaker
    - AND the message is emotionally complex (detected via sentiment length,
      reaction words, or a confidence threshold)

    Uses a cheap 1B parameter model or all-MiniLM classifier.
    """
    prompt = f"""Given this conversation, who is the user most likely addressing?

Recent context:
{format_recent_history(conversation_history, n=6)}

User's message: "{user_message}"

Reply with exactly one word: 'kelly', 'nathan', or 'both'.
If genuinely ambiguous, reply 'last' (use last speaker)."""

    result = quick_model.generate(prompt, max_tokens=5, temperature=0.1)
    label = result.strip().lower()

    if label in ('kelly', 'nathan', 'both'):
        return label
    # 'last' or unparseable → fall through to heuristic
    return None
```

When to enable:
- v1: disabled. Heuristic + forced alternation is sufficient for launch.
- v2+: enable if users report "wrong routing" in emotional/complex scenes.
- Can be gated behind a per-chat toggle: `use_llm_router: false` (default off).

Why not v1: The heuristic handles 90%+ of cases correctly. The LLM router adds latency (~200-500ms) and cost per ambiguous message. It's a refinement, not a foundation — ship the heuristic first, measure misrouting rates, then add the router only if needed.

Model choice: A 1B parameter model (e.g., Qwen2.5-1.5B, Llama-3.2-1B) or a sentence classifier (all-MiniLM fine-tuned on a small routing dataset). The task is simple classification — no need for a large model. Can run locally via Ollama to avoid API costs.

Ambient Responses (Alternative Tier 4 Default):

Instead of "who is this message for?" the default question can be: "Both characters heard this. Who speaks first, and does the other add anything?"

When `ambient_responses: true` (config toggle, default `false`), ambiguous Tier 4 messages default to a **group response** rather than single-character routing:

1. Last speaker responds first (they get first crack at the ambiguous message)
2. The other character gets a short-response call: "Kelly just said [X] in response to the user saying '[ambiguous message]'. You're present. React naturally or stay quiet."

This means ambiguous messages default to a group response rather than a single-character response. It's more expensive (two calls instead of one), but it's more believable — both characters are present for every ambiguous message.

When to enable: Off by default for users who want faster exchanges. Enable for users who prioritize the "hanging out" feel over speed. The passive presence system (§6.6) already covers most of this ground — ambient responses is a more aggressive version where *every* ambiguous message gets a group response, not just a probability-based reaction.

Relationship to passive presence: If `ambient_responses: true` AND `passive_presence_enabled: true`, the passive reaction check is skipped on Tier 4 turns (the other character already responded via the ambient response). On Tier 1-3 turns (explicit routing), passive presence still fires normally.

5.6 Example Walkthrough

```
[You]: What was it like? Reading his mind?
    → Tier 3: "reading" + "mind" → Kelly (content inference)
    → consecutive_default_turns unchanged (content inference doesn't increment)
    → Route to Kelly

[Kelly]: It were horrible, actually. His thoughts were... loud.
    → last_speaker = kelly

[You]: Yeah but what did he think about?
    → Tier 1: no names found
    → Tier 2: no "both" keywords
    → Tier 3: "think" → Kelly keyword match
    → Route to Kelly (content inference)

[Kelly]: Mostly about me. Which were weird.
    → last_speaker = kelly

[You]: That's mad.
    → Tier 1: no names
    → Tier 2: no "both"
    → Tier 3: no keyword match
    → Tier 4: last_speaker = kelly, consecutive_default_turns[kelly] = 0 < 2
    → Route to Kelly, increment: consecutive_default_turns[kelly] = 1

[Kelly]: Proper mad. He was obsessed.
    → last_speaker = kelly

[You]: Hmm.
    → Tier 4: last_speaker = kelly, consecutive_default_turns[kelly] = 1 < 2
    → Route to Kelly, increment: consecutive_default_turns[kelly] = 2

[Kelly]: Yeah.
    → last_speaker = kelly

[You]: Go on then.
    → Tier 4: last_speaker = kelly, consecutive_default_turns[kelly] = 2 >= max (2)
    → FORCE SWITCH to Nathan
    → Reset both counters to 0
    → Route to Nathan

[Nathan]: —oh NOW you want to hear from me? Cheers for that.
    → last_speaker = nathan

[You]: Kel... you know that's not true.
    → Tier 1: word-boundary match: \bkel\b → Kelly (nickname)
    → Reset consecutive_default_turns for both to 0
    → Route to Kelly

[Kelly]: I know. He's just being a dick.
    → last_speaker = kelly

[You]: Nathan, was that true?
    → Tier 1: "Nathan" found → Nathan
    → Reset consecutive_default_turns for both to 0
    → Route to Nathan

[Nathan]: —oh she's being dramatic. I barely thought about her at all.
    → last_speaker = nathan

[You]: You two are exhausting.
    → Tier 2: "you two" → Both
    → Route to both (sequential)
```

---

6. Interruption System

6.1 Overview

This is the feature that makes the conversation feel *alive*. Characters don't just take turns politely — they cut each other off, talk over each other, and react in real-time.

Two modes of interruption:

Mode	When Used	Mechanism
Scripted Interruptions	"Let them talk" mode	Single API call, model writes both characters with `—` em-dash protocol
Live Interruptions	User-directed turns	Streaming interrupt monitor watches Character A, triggers Character B mid-stream

6.2 Em-Dash Protocol (Shared Convention)

Both modes use the same textual convention for interruptions:

```
[Kelly]: I could hear everything he was thinking, and it were all just—
[Nathan]: —all about me? Yeah, I know. I'm unforgettable.
```

- **Trailing** `—` on a message = "I was cut off"
- **Leading** `—` on the next message = "I'm picking up from where you were cut off"

This is the standard convention used in novels and screenplays. The model is instructed to use it in the combo prompt rules.

6.3 Mode 1: Scripted Interruptions (Let Them Talk)

Used when the user clicks "Let them talk" — characters exchange 1-N rounds without user input.

Single API call. The model generates the full scene, writing both characters' dialogue with natural interruptions. The output is a single text block with alternating character labels:

```
[Kelly]: I'm just saying, it's weird that you can't die. It's not natural.
[Nathan]: Oh, and hearing people's thoughts is TOTALLY natural? At least my
          thing is just... not dying. Yours is a proper violation of—
[Kelly]: —don't you dare finish that sentence.
[Nathan]: —privacy. I was going to say privacy. God, you're paranoid.
[Kelly]: I can literally hear what you're thinking right now, and you were
         NOT going to say privacy.
[Nathan]: <<YIELD>>
```

Parsing: The backend splits the response on `[Name]:` labels, creating separate message objects. Each segment becomes a message with the appropriate `speaker` field. Trailing/leading em-dashes are preserved in content and flagged with `interrupted: true` / `is_interruption: true`.

Yield token: `<<YIELD>>` signals "this character is done and yielding control back." When the parser encounters it, the exchange ends — even if fewer than `let_them_talk_rounds` have completed. The model controls the natural endpoint.

Round limiting: The combo prompt includes: "Exchange at most N rounds. When done, emit <>." The backend also enforces a hard cap — if the model doesn't yield after N rounds, the stream is cut.

6.4 Mode 2: Live Interruptions (Streaming)

Used during user-directed turns. Character A is streaming their response. Character B may interrupt mid-stream.

Architecture:

Interrupt Triggers (in priority order):

1. **Explicit Yield (** `<<YIELD>>` **)** — Character A's model emits the yield token. This is the model saying "I'm pausing for a reaction." Highest priority — always honored.
2. **Keyword Trigger** — Character B has `interrupt_keywords` defined. If any keyword appears in A's streaming text, B interrupts. Example: Nathan's keywords include "community service" — if Kelly says "community service," Nathan cuts in.
3. **Patience Threshold** — Character B has a `patience_threshold` (character count). If A's stream exceeds this length without yielding, B auto-interrupts. Nathan's threshold is 150 chars (no patience). Kelly's is 300 (more patient).

Interrupt Flow (detailed):

```
1. A is streaming. Monitor detects trigger at character position N.

2. Backend sends SSE event:
   {"event": "interrupt", "by": "nathan", "truncated_at": N}

3. Frontend:
   - Appends "—" to A's current bubble
   - Marks A's message as interrupted: true
   - Creates new bubble for Nathan
   - Scrolls to new bubble

4. Backend:
   - Appends A's truncated message to shared history[]
   - Starts B's API call with augmented system context:
     "Kelly was mid-sentence saying: '[truncated text]—'.
      You interrupted her. Respond as Nathan, picking up from
      the interruption. Begin with — to signal the cut-in."

5. B streams. Monitor watches B's stream for A's interrupt triggers
   (roles are now reversed).

6. After B completes (no interrupt from A):
   - Backend sends SSE: {"event": "resume_available", "for": "kelly"}
   - Frontend shows "Kelly may respond" indicator
   - Backend makes A's "resume" call:
     "You were interrupted by Nathan. He said: '[B's full response]'.
      You may continue your thought, respond to his interruption,
      or yield by emitting <<YIELD>>."

7. If A yields → SSE: {"event": "turn_complete"}
   If A continues → stream resumes, cycle repeats
```

Guard Rails:

- **Max interrupts per turn:** 3 (prevents infinite ping-pong). After 3 interrupts, the current speaker finishes without interruption.
- **Cooldown:** After an interrupt, a 500-char cooldown before the next interrupt can fire (prevents rapid-fire cutting).
- **User override:** User can type during streaming to force-stop all characters (existing abort behavior).

6.5 Frontend Interrupt Visualization

```
┌──────────────────────────────────────────┐
│  Kelly Bailey                            │
│ I could hear everything he was thinking, │
│ and it were all just—                    │  ← truncated, em-dash
├──────────────────────────────────────────┤
│    Nathan Young                          │
│ —all about me? Yeah, I know. I'm         │  ← leading em-dash (cut-in)
│ unforgettable.                           │
├──────────────────────────────────────────┤
│    Kelly Bailey                          │
│ I'm going to kill him.                   │  ← resume after interrupt
└──────────────────────────────────────────┘
```

Visual cues:
- Interrupted messages get a **dashed right border** (instead of solid)
- Interrupting messages get a **dashed left border**
- A subtle **zigzag connector** animation can play between the two bubbles
- The participant strip highlights the active speaker during streaming

6.6 Passive Presence System (The Missing Character)

The Problem: When the addressing resolver routes to Kelly, Nathan ceases to exist until either the interrupt monitor fires or the user explicitly addresses him. The conversation becomes a series of 1-on-1 exchanges spliced together with a shared history. That's not hanging out with two people — that's alternating between two separate conversations.

After 20 turns, the user feels the pattern: say something → one character responds → say something → one character responds. The other person is a ghost until summoned. That's a queue, not a group dynamic.

The Solution: After the addressed character responds, roll a probability check for the non-addressed character. Not a full response — a **reaction**. One line, maybe two. A snort, an eye-roll, a muttered comment, a look exchanged with the other character, pulling out a phone because they're bored. Or silence.

```
def maybe_passive_reaction(addressed_speaker, other_participant, conversation_context,
                           emotional_state, config):
    """After the addressed character finishes, the other character may react.

    This is NOT a full response. It's a brief, uninvited reaction that
    keeps the non-speaking character present in the scene.
    """
    if not config.get('passive_presence_enabled', True):
        return None

    # Roll against the character's reaction_probability
    # Modulated by emotional state (§6.7)
    base_prob = other_participant.get('reaction_probability', 0.3)
    energy = emotional_state[other_participant['label']]['energy']

    # Low energy → less likely to react
    energy_mod = {'low': -0.15, 'medium': 0.0, 'high': +0.15}.get(energy, 0.0)
    actual_prob = max(0.0, min(0.9, base_prob + energy_mod))

    if random.random() > actual_prob:
        return None  # Stays quiet this turn

    # Brief reaction call — lightweight, max 60-80 tokens
    prompt = f"""{other_participant['display_name']} just overheard this exchange.
{addressed_speaker} was addressed and said: \"{conversation_context['last_response']}\".

You weren't addressed. React BRIEFLY (1-2 sentences max) if {other_participant['label']}
would naturally react — a comment, a snort, a look, a mutter, a deflection.

If {other_participant['label']} would genuinely stay quiet, respond with <<SILENT>>.

Current emotional state: {emotional_state[other_participant['label']]}
"""

    result = quick_generate(
        model=config['trio_model'],
        system=build_character_focused_prompt(other_participant, summary_only=True),
        prompt=prompt,
        max_tokens=other_participant.get('reaction_max_tokens', 80),
        temperature=0.7
    )

    if '<<SILENT>>' in result:
        return None  # Model decided the character would stay quiet

    return {
        'speaker': other_participant['label'],
        'content': result.strip(),
        'is_passive_reaction': True,  # flag for UI styling
        'max_tokens': other_participant.get('reaction_max_tokens', 80)
    }
```

Reaction probability per character:

Character	Base Probability	Rationale
Nathan Young	0.60	Can't shut up, needs to comment on everything, craves attention
Kelly Bailey	0.35	More guarded, reacts when something actually warrants it

Emotional modulation: The reaction probability is modulated by the character's current `energy` level (§6.7). If Nathan's energy drops to "low" after a serious moment, his reaction probability drops from 60% to 45%. If Kelly's energy is "high" (she's fired up), hers rises from 35% to 50%.

The `<<SILENT>>` token: Even when the probability roll hits, the model may decide the character would genuinely stay quiet. The `<<SILENT>>` token lets the model opt out. This means the probability is an *upper bound* on reactivity, not a guarantee — the character's personality is the final arbiter.

UI rendering: Passive reactions are styled differently from full responses:
- Smaller font size (90%)
- Muted opacity (0.7)
- No avatar/header — just the text with a subtle left-border in the character's color
- A small "reacted" indicator (e.g., a subtle icon or italic styling)

This visually communicates: "this is a background reaction, not a main response."

Cost: One additional lightweight API call per turn (max 80 tokens, ~0.5-1 second latency). Can be made parallel with the main response's interrupt monitoring. Gated behind `passive_presence_enabled: true`.

Interaction with trio state (CRITICAL):

Passive reactions have specific rules for how they interact with routing state:

State	Updated by passive reaction?	Why
`last_speaker`	NO	If Nathan's passive snort set `last_speaker = nathan`, the user's next vague "lol" would route to Nathan via Tier 4 — the passive presence system would silently hijack routing. `last_speaker` only updates on full responses.
`consecutive_default_turns`	NO	Passive reactions don't count as "turns" for alternation purposes. They're background color, not conversational turns.
`history[]`	YES	Passive reactions ARE appended to shared history (with `is_passive_reaction: true` flag). If they weren't, characters would contradict things they visibly said. The model sees them in context.
`emotional_state`	YES	A passive reaction can shift emotional state (Nathan's snort might annoy Kelly). The emotional state update call runs after passive reactions.

Summary: Passive reactions are appended to history (so characters remember them) but do NOT update `last_speaker` or `consecutive_default_turns` (so they don't hijack routing). This must be explicit in the trio_state spec and the turn flow (§7.2).

Why this transforms the experience: The user no longer feels like they're talking to one person at a time. Even when they address Kelly, Nathan might snort, mutter "here we go," or exchange a look. The possibility that the other character *might* react keeps them present in the user's mind. This is the difference between "multi-character chat" and "being in a room with two people."

6.7 Emotional State Tracking

The Problem: Characters start with a static emotional state (from the relationship dynamic config) and never change. After 30 turns, Kelly is still "guarded" and Nathan is still "performing" regardless of what happened. Real people's moods shift based on conversation — a serious moment drops Nathan's energy, a joke raises Kelly's.

The Solution: A lightweight per-character mood tracker that updates after each exchange and injects into the combo prompt's dynamic section, replacing the static `current_tension` field with living emotional context.

```
emotional_state = {
    'kelly': {
        'current': 'guarded',        # overall mood: guarded, open, angry, amused, vulnerable...
        'toward_nathan': 'annoyed',  # how Kelly feels about Nathan right now
        'toward_user': 'warming_up', # how Kelly feels about the user
        'energy': 'low'              # low | medium | high — affects reaction probability, response length
    },
    'nathan': {
        'current': 'performing',
        'toward_kelly': 'prodding',
        'toward_user': 'showing_off',
        'energy': 'high'
    }
}
```

Update mechanism: After each exchange (main response + any passive reactions), a fast call updates the state:

```
def update_emotional_state(speaker, exchange_summary, current_state, config):
    """Update a character's emotional state after an exchange.

    Lightweight call — the model reads what just happened and adjusts
    the character's internal state. This state feeds back into the next
    combo prompt's PRESENCE & EMOTIONAL CONTEXT section.
    """
    prompt = f"""Given what just happened in the exchange below, how does {speaker} feel now?

Exchange summary: {exchange_summary}

Current state: {current_state}

Update {speaker}'s emotional state. Respond in JSON:
{{
    "current": "<one word: guarded|open|angry|amused|vulnerable|performing|deflecting|serious|playful>",
    "toward_other": "<one word: annoyed|fond|protective|prodding|trusting|suspicious|indifferent>",
    "toward_user": "<one word: warming_up|trusting|suspicious|amused|bored|engaged|defensive>",
    "energy": "<low|medium|high>"
}}

Only change fields that would actually shift given this exchange.
If nothing would change, return the current state unchanged.
"""
    result = quick_generate(
        model=config['trio_model'],
        prompt=prompt,
        max_tokens=60,
        temperature=0.3  # low temp for consistency
    )
    return parse_json(result)
```

What this enables:

1. **Character arcs within a conversation.** Nathan starts "performing" (high energy, showing off). After a serious moment with Kelly, his state shifts to "serious" (medium energy). His responses get quieter, less jokey. That arc is what makes characters feel real.
2. **Modulated passive reactions.** The passive presence system (§6.6) uses `energy` to adjust reaction probability. A tired Nathan reacts less. A fired-up Kelly reacts more.
3. **Dynamic combo prompt.** The `PRESENCE & EMOTIONAL CONTEXT` section in the combo prompt (§4.1) pulls from this state instead of a static `current_tension` field. The model knows: "Nathan is currently serious, his energy is low, he's feeling protective toward Kelly." This informs how Kelly responds to the user — she might soften because Nathan's being uncharacteristically genuine.
4. **Interrupt motivation.** When the interrupt system (§6.4) checks whether Nathan should interrupt, the emotional state provides context. A "performing" Nathan with "high" energy interrupts to riff. A "serious" Nathan with "low" energy might let Kelly finish.

Cost: One additional lightweight API call per exchange (~60 tokens, ~0.5 second). Can run in parallel with the passive reaction check. Gated behind `emotional_tracking_enabled: true`.

Update frequency: After every exchange (main response + passive reactions). Not after every token — that would be excessive. The state represents the character's mood *between* turns, not mid-sentence.

6.8 The "Both Hear Everything" Principle

The Problem: Even with shared message history, the model needs to be explicitly reminded that both characters are present at all times. Without this, Kelly's responses read as if Nathan doesn't exist until he speaks.

The Solution: The combo prompt's `PRESENCE & EMOTIONAL CONTEXT` section (added in §4.1) explicitly states:

```
Both characters are present in the scene. They hear everything.
Nathan is here. His current state: <emotional_state.nathan>
Kelly is here. Her current state: <emotional_state.kelly>
The character NOT speaking may react briefly after the speaker finishes.
```

This lets the addressed character's responses naturally reference the other's presence:

- Kelly: "Nathan's sitting there looking smug, ignore him."
- Nathan: "Kelly's about to tell you to shut me up. Aren't you, Kel?"
- Kelly (addressed): "Don't look at him. He's not involved in this."

The model acknowledges the other character exists even when they're not the one talking. That's presence.

Combo prompt rule addition (§4.1 rule 9):

```
9. Both characters are ALWAYS present in the scene. They hear everything said.
   Even when only one character is speaking, the other is there — listening,
   reacting internally, maybe about to speak. The speaking character may
   reference the other's presence naturally. Do not act as if the non-speaking
   character has vanished.
```

6.9 Motivation-Based Interrupts (v2+ Enhancement)

The Problem with v1 interrupts: The current interrupt system (§6.4) uses keyword triggers and patience thresholds. These are mechanical — Nathan interrupts because Kelly hit 150 characters or said "community service." By the tenth interrupt, the user feels the mechanism: "Oh, Kelly hit 150 characters, here comes Nathan." The interrupts feel like a timer, not a person who can't help themselves.

The v2+ solution: Replace mechanical triggers with a motivation-based classification call. After every N tokens of Character A's stream, a fast classification determines whether Character B would *want* to interrupt:

```
def check_interrupt_motivation(streamed_text, other_character_config, emotional_state):
    """v2+: Motivation-based interrupt check.

    Replaces keyword/patience triggers with an emotional classification.
    The model determines whether Character B would naturally interrupt
    given their personality and the current emotional context.
    """
    prompt = f"""{other_character_config['display_name']} is watching {streamed_text[:200]}...

{other_character_config['display_name']}'s personality: {other_character_config['summary']}
Current emotional state: {emotional_state}

Would {other_character_config['label']} interrupt right now? Answer with one:
- YES_CANT_RESIST (must say something, can't hold back)
- YES_DISAGREES (actively objects to what's being said)
- YES_DEFLECTING (uncomfortable, needs to redirect)
- LEANING_IN (wants to but holding back — do NOT interrupt yet)
- NO (listening, no urge to interrupt)
"""
    result = quick_generate(model=config['trio_model'], prompt=prompt,
                            max_tokens=10, temperature=0.2)
    return result.strip().upper()
```

v1 approach (keep for launch): Keyword + patience threshold. It's fast, free, and produces acceptable results for the first version. The patience threshold remains as a **hard ceiling** in v2+ — if Character A monologues past 500+ characters, the other character cuts in regardless of motivation.

When to upgrade: If users report that interrupts feel "mechanical" or "timer-like" after extended sessions, enable the motivation-based system. It's gated behind `motivation_based_interrupts: false` by default.

---

7. Turn Management

7.1 Turn Modes

Mode	Description	When to Use
`context` (default)	Addressing resolver determines who responds. Single character by default, both if ambiguous or "both" detected.	Normal conversation flow
`sequential`	Both characters always respond in order (A then B).	User wants both perspectives on everything
`simultaneous`	Both characters stream at the same time in parallel.	Quick reactions, neither influenced by the other. **Deferred to v2+** — requires parallel streaming, history-ordering rules, and interrupt interaction spec that aren't in v1.
`let_them_talk`	No user input. Characters exchange N rounds on their own.	User wants to watch them interact

7.2 Turn Flow: Context Mode (Default)


User sends message
Addressing Resolver (§5)
@Kelly only
Single API call (Kelly responds, streaming)
Interrupt monitor watches (§6.4)
No interrupt → Kelly finishes
Nathan interrupts → Nathan responds → Kelly may resume
★ Passive reaction check (§6.6)
Roll for Nathan's reaction_probability (modulated by energy)
Roll fails or <<SILENT>> → Nathan stays quiet
Roll hits → Nathan reacts briefly (1-2 sentences)
★ Emotional state update (§6.7)
Update Kelly's and Nathan's emotional_state
Return to user
@Nathan only
(mirror of above, roles swapped)
Both
Call 1: Kelly responds → appended to history
Call 2: Nathan responds (sees Kelly's response) → appended
★ Emotional state update (both characters)
Return to user
(No passive reaction — both already spoke)
Last speaker (default, Tier 4)
If ambient_responses: true (§5.5)
Both respond: last_speaker first, then other reacts
(This is the "group response" default — see §5.5)
If ambient_responses: false (default)
Single API call (last_speaker responds)
Interrupt monitor watches
No interrupt → finish
Other interrupts → exchange → resume
★ Passive reaction check (§6.6)
★ Emotional state update (§6.7)

```
                ▼
            Return to user
```

Key additions to the turn flow:

1. **Passive reaction check (★)** — After the addressed character finishes (and any interrupt exchange resolves), the non-speaking character gets a probability-based reaction roll. This happens on every single-character turn. It does NOT happen on "both" turns (both already spoke) or "let them talk" turns (the model handles presence natively in the combo prompt).
2. **Emotional state update (★)** — After the exchange completes (main response + passive reaction), both characters' emotional states are updated via a lightweight call. This feeds into the next turn's combo prompt.
3. **Ambient responses (Tier 4)** — When `ambient_responses: true`, ambiguous messages default to a group response instead of single-character routing. See §5.5 for details.

7.3 Turn Flow: Let Them Talk

```
User clicks "Let them talk"
       │
       ▼
Backend builds combo prompt with:
  "Current turn: Let them talk
   Exchange at most N rounds. Write both characters.
   Use [Name]: labels. Use — for interruptions.
   Emit <<YIELD>> when the exchange naturally concludes."
       │
       ▼
Single API call (streaming)
       │
       ▼
Backend parses stream in real-time:
  - Split on [Name]: labels
  - Each segment → SSE event with speaker tag
  - <<YIELD>> → end stream
  - N rounds reached → end stream
       │
       ▼
Frontend renders each segment as a separate message bubble
with appropriate speaker, color, and interrupt flags
       │
       ▼
All messages appended to shared history[]
       │
       ▼
Return to user
```

---

8. Backend Implementation

8.1 stream_chat Trio Branch

New branch in `routes/chat.py` `stream_chat()`:

```
if config.get('mode') == 'trio':
    participants = config['participants']
    combo_prompt = build_combo_prompt(participants, config.get('dynamic', {}))

    # Run per-character lore/RAG injection
    for p in participants:
        char_name = p['config'].replace('.json', '')
        lore_msgs = lore_scan(messages, char_name, char_name, ...)
        # Inject into combo prompt's character section

    # Addressing resolution
    addressee = resolve_addressee(user_message, participants, trio_state)

    if addressee == 'both':
        # Sequential calls
        for p in participants:
            yield from stream_character_response(p, combo_prompt, messages, ...)
            # Each call appends to shared messages[]
    elif addressee == 'let_them_talk':
        # Single call, dual-response
        yield from stream_let_them_talk(combo_prompt, messages, config, ...)
    else:
        # Single character
        p = next(p for p in participants if p['label'] == addressee)
        yield from stream_character_response_with_interrupts(
            p, combo_prompt, messages, participants, config, ...
        )
```

8.2 Combo Prompt Builder

```
def build_combo_prompt(participants, dynamic):
    """Build structured combo system prompt from participant configs."""
    sections = []

    for i, p in enumerate(participants, 1):
        char_config = load_character_config(p['config'])
        label = p['display_name']
        sections.append(f"=== CHARACTER {i}: {label} ===")
        sections.append(char_config.get('system_prompt', ''))
        sections.append(char_config.get('character_card', ''))
        # Personality/speech pattern notes
        sections.append(char_config.get('speech_patterns', ''))

    # Dynamic section
    sections.append("=== RELATIONSHIP DYNAMIC ===")
    for p in participants:
        other = next(o for o in participants if o != p)
        rel = dynamic.get(f"{p['label']}_to_{other['label']}", '')
        if rel:
            sections.append(f"{p['display_name']} → {other['display_name']}: {rel}")
    if dynamic.get('shared_history'):
        sections.append(f"Shared history: {dynamic['shared_history']}")
    if dynamic.get('current_tension'):
        sections.append(f"Current tension: {dynamic['current_tension']}")

    # Rules section
    sections.append("=== CONVERSATION RULES ===")
    sections.append(RULES_TEMPLATE)  # Pre-defined rules text

    # Turn indicator (filled per-call)
    sections.append("=== TURN INDICATOR ===")
    # Filled by caller

    return "\n\n".join(sections)
```

8.3 Addressing Resolver

```
import re

def resolve_addressee(user_message, participants, trio_state, max_consecutive=2):
    """Determine which character(s) the user is addressing.

    Four-tier resolution:
      1. Name detection (word-boundary + prefix fallback, @ optional)
      2. "Both" keywords
      3. Content inference (addressing_keywords per character)
      4. Last-speaker default with forced alternation

    Returns: 'kelly', 'nathan', 'both', or 'let_them_talk'
    """
    msg_lower = user_message.lower()
    # Strip @ symbols — they're optional, just a UI hint
    msg_clean = msg_lower.replace('@', '')
    consecutive = trio_state.get('consecutive_default_turns', {})

    # ── Tier 1: Name Detection ──────────────────────────────
    name_matches = _detect_names(msg_clean, participants)

    if not name_matches:
        # Prefix fallback for typos, trailing vowels, ellipsis
        name_matches = _detect_names_prefix(msg_clean, participants)

    if name_matches:
        if len(name_matches) == 1:
            # Single character named → route to them
            _reset_consecutive(consecutive)
            return name_matches[0]['label']
        else:
            # Both characters named → first-mentioned is addressee (vocative)
            name_matches.sort(key=lambda m: m['position'])
            _reset_consecutive(consecutive)
            return name_matches[0]['label']

    # ── Tier 2: "Both" Keywords (word-boundary matched) ─────────────────
    # Use word-boundary regex, not substring, to prevent "both" matching
    # inside "bothered" or "you two" matching inside "you twosome".
    both_keywords = ['both', 'you two', 'you guys', 'both of you', 'you both']
    for kw in both_keywords:
        pattern = r'\b' + re.escape(kw) + r'\b'
        if re.search(pattern, msg_clean):
            _reset_consecutive(consecutive)
            return 'both'

    # ── Tier 3: Content Inference (word-boundary matched) ────────────────
    # Same word-boundary fix as Tier 2. Prevents "hear" matching "heart",
    # "die" matching "audience", "mind" matching "reminded", etc.
    keyword_matches = {}
    for p in participants:
        char_config = load_character_config(p['config'])
        keywords = char_config.get('addressing_keywords', [])
        score = 0
        for kw in keywords:
            pattern = r'\b' + re.escape(kw) + r'\b'
            if re.search(pattern, msg_clean):
                score += 1
        if score > 0:
            keyword_matches[p['label']] = score
            keyword_matches[p['label']] = score

    if len(keyword_matches) == 1:
        # NOTE: Content inference does NOT reset consecutive counter.
        # It's a soft signal — only explicit name/both routing resets it.
        return list(keyword_matches.keys())[0]
    # If both match or neither matches, fall through

    # ── Tier 4: Last-Speaker Default with Forced Alternation ────────
    last = trio_state.get('last_speaker', participants[0]['label'])
    last_count = consecutive.get(last, 0)
```

```
    if last_count >= max_consecutive:
        # Force switch to the other character
        other = next(p['label'] for p in participants if p['label'] != last)
        consecutive[last] = 0
        consecutive[other] = 0
        return other
    else:
        # Route to last speaker, increment their counter
        consecutive[last] = last_count + 1
        return last


def _detect_names(msg_clean, participants, config_cache=None):
    """Word-boundary regex match for character names and nicknames.

    Uses config_cache (dict of config_path → char_config) to avoid
    reloading character configs from disk on every message. The cache
    is built once per chat session and passed through the resolver.
    """
    matches = []
    for p in participants:
        char_config = _get_cached_config(p['config'], config_cache)
        names = char_config.get('name_variants', [p['display_name']])
        for name in names:
            name_lower = name.lower()
            pattern = r'\b' + re.escape(name_lower) + r'\b'
            m = re.search(pattern, msg_clean)
            if m:
                matches.append({
                    'label': p['label'],
                    'name_matched': name,
                    'position': m.start()
                })
                break  # one match per character is enough
    return matches


def _get_cached_config(config_path, cache):
    """Get character config from cache, loading from disk only on first access."""
    if cache is None:
        return load_character_config(config_path)
    if config_path not in cache:
        cache[config_path] = load_character_config(config_path)
    return cache[config_path]


def _detect_names_prefix(msg_clean, participants):
    """Prefix fallback: handles 'Kellyyy', 'Kell' (typo), 'Kel...' (trailing).

    Guards against false positives like 'kelp' matching 'kel':
    - Requires the matched word be at most len(name) + 3 characters
      (allows 'Kellyyy' but not 'kelvin' for 'Kel')
    - Only runs on short messages (< 100 chars) where a vocative is plausible
    """
    if len(msg_clean) > 100:
        return []  # prefix fallback only for short, vocative-style messages

    words = re.findall(r'\b\w+\b', msg_clean)
    matches = []
    for word in words:
        for p in participants:
            char_config = load_character_config(p['config'])
            names = char_config.get('name_variants', [p['display_name']])
            for name in names:
                name_lower = name.lower()
                if len(name_lower) >= 3 and word.startswith(name_lower):
                    # Guard: word must be close to the name length
                    # 'Kellyyy' (7) vs 'Kelly' (5) → OK (diff = 2)
                    # 'kelvin' (6) vs 'Kel' (3) → rejected (diff = 3)
                    if len(word) - len(name_lower) > 3:
                        continue
                    matches.append({
                        'label': p['label'],
```

```
                        'name_matched': name,
                        'position': msg_clean.index(word)
                    })
                    break
            else:
                continue
            break
    return matches


def _check_trailing_vocative(msg_clean, participants):
    """Check if message ends with a vocative: ', Kelly' or 'Kelly?'

    Trailing vocatives override first-mentioned position routing.
    "Tell Nathan he's wrong, Kelly" → Kelly is the addressee, not Nathan.
    """
    msg_stripped = msg_clean.rstrip('?!.,')
    for p in participants:
        char_config = load_character_config(p['config'])
        names = char_config.get('name_variants', [p['display_name']])
        for name in names:
            name_lower = name.lower()
            if msg_stripped.endswith(', ' + name_lower) or msg_stripped.endswith(name_lower):
                return p['label']
    return None


def _reset_consecutive(consecutive):
            break
    return matches


def _reset_consecutive(consecutive):
    """Reset consecutive default-turn counters for all characters."""
    for key in list(consecutive.keys()):
        consecutive[key] = 0
```

Counter reset rules (summary):

Event	Resets `consecutive_default_turns`?
Tier 1: Name detected	Both characters reset to 0
Tier 2: "Both" keyword	Both characters reset to 0
Tier 3: Content inference	No reset (soft signal)
Tier 4: Last-speaker default	No reset (increments routed char)
Tier 4: Forced alternation	Both reset to 0 (new streak)
Let-them-talk exchange	Both reset to 0 (flow changed)

8.4 Interrupt Monitor

```
class InterruptMonitor:
    """Watches a character's streaming output for interrupt triggers.

    Patience triggers are PROBABILISTIC, not deterministic. When the patience
    threshold is reached, a probability roll determines whether the interrupt
    fires. This prevents the mechanical "every long response gets interrupted"
    pattern. Additionally, consecutive patience-interrupts decay in probability
    so the pattern doesn't repeat identically each turn.
    """

    def __init__(self, other_character_config, max_interrupts=3, cooldown_chars=500,
                 patience_probability=0.40, patience_decay=0.15):
        self.other = other_character_config
        self.keywords = other_character_config.get('interrupt_keywords', [])
        self.patience = other_character_config.get('patience_threshold', 300)
        self.patience_probability = other_character_config.get(
            'patience_interrupt_probability', patience_probability
        )
        self.patience_decay = patience_decay  # each consecutive patience-interrupt reduces probability
        self.max_interrupts = max_interrupts
        self.cooldown_chars = cooldown_chars
        self.interrupt_count = 0
        self.consecutive_patience_interrupts = 0  # for decay calculation
        self.chars_since_last_interrupt = 0
        self.buffer = ""

    def check(self, new_text):
        """Check if an interrupt should fire. Returns trigger type or None."""
        if self.interrupt_count >= self.max_interrupts:
            return None

        if self.chars_since_last_interrupt < self.cooldown_chars and self.interrupt_count > 0:
            return None

        self.buffer += new_text
        self.chars_since_last_interrupt += len(new_text)

        # 1. Explicit yield — always fires (model chose to yield)
        if "<<YIELD>>" in self.buffer:
            self.consecutive_patience_interrupts = 0  # reset decay
            return 'yield'

        # 2. Keyword trigger — always fires (content-driven, not mechanical)
        buffer_lower = self.buffer.lower()
        for kw in self.keywords:
            if kw in buffer_lower:
                self.consecutive_patience_interrupts = 0  # reset decay
                return 'keyword'

        # 3. Patience threshold — PROBABILISTIC
        # Roll a probability check. Decay reduces the chance for each
        # consecutive patience-triggered interrupt, preventing the
        # "every long response gets cut off" pattern.
        if len(self.buffer) >= self.patience:
            effective_prob = max(0.05, self.patience_probability -
                                 (self.consecutive_patience_interrupts * self.patience_decay))
            if random.random() < effective_prob:
                self.consecutive_patience_interrupts += 1
                return 'patience'
            else:
                # Didn't fire — reset buffer to prevent re-rolling every token
                # Keep last 50 chars for context, discard the rest
                self.buffer = self.buffer[-50:]

        return None

    def fire(self):
        """Mark an interrupt as fired."""
        self.interrupt_count += 1
        self.chars_since_last_interrupt = 0
        self.buffer = ""
```

8.5 SSE Protocol Extensions

New SSE event types for trio mode. **Critical:** the frontend never receives raw model tokens. The backend Token Buffer Queue (§8.6) assembles all structural markers before emitting these structured events:

```
// Normal token stream — speaker already resolved by backend
{"speaker": "kelly", "delta": "I could hear "}

// Interrupt event — backend detected trigger, truncated A, starting B
{"event": "interrupt", "by": "nathan", "truncated_text": "I could hear everything he was thinking, and it
were all just—"}

// Resume available
{"event": "resume_available", "for": "kelly"}

// Turn complete
{"event": "turn_complete"}

// Let-them-talk segment — backend parsed [Name]: labels, emits clean segments
{"speaker": "kelly", "delta": "I'm just saying, ", "segment": 1}
{"speaker": "nathan", "delta": "Oh, and hearing people's thoughts is ", "segment": 2}
```

8.6 Token Buffer Queue (Critical: Prevents Mid-Token Parsing Failure)

The Problem: Modern LLMs emit tokens as fractions of words. A single `[Kelly]:` label might arrive as three separate chunks: `[K`, then `elly]:`, then `It were...`. If the backend forwards raw chunks directly to the frontend, the JavaScript string parser (`delta.includes("[Kelly]:")`) will break mid-token — it'll never see the complete label and won't know which speaker the text belongs to.

The same problem applies to `<<YIELD>>` tokens (might arrive as `<<YIE`, `LD>>`) and em-dash interrupt markers.

The Solution: The backend must consume tokens internally through a buffer queue until it can confidently determine whether a structural marker has fully formed. Only then does it emit structured SSE events with pre-resolved `speaker` fields. The frontend never does string parsing on raw tokens.

```
class TokenBufferQueue:
    """Buffers raw model tokens, emits structured SSE events once
    structural markers ([Name]:, <<YIELD>>) fully form.

    The frontend NEVER receives raw tokens — only structured events
    with pre-resolved speaker fields.

    Note: em-dash interrupt markers are NOT parsed here. Em-dashes are
    emitted as part of the interrupt event's truncated_text field by the
    InterruptMonitor (§8.4), not as a structural marker in the stream.
    """

    YIELD_TOKEN = '<<YIELD>>'

    def __init__(self, participants, default_speaker=None):
        """Initialize buffer queue.

        Args:
            participants: List of participant config dicts.
            default_speaker: The label of the addressed character. Required
                for single-character streams where the model won't emit
                [Name]: labels. If None and no label appears, all text
                is dropped — so always pass this for non-let-them-talk calls.
        """
        self.participants = participants
        self.default_speaker = default_speaker
        self.current_speaker = default_speaker  # start with default, not None
        self.buffer = ""
        self.pending_yield = False

        # Pre-build label → speaker map and regex from KNOWN labels.
        # This avoids disk reads during streaming and matches display names.
        self._label_map = {}  # "kelly" → "kelly", "kelly bailey" → "kelly"
        for p in participants:
            self._label_map[p['label'].lower()] = p['label']
            self._label_map[p['display_name'].lower()] = p['label']
            # Pre-load name_variants too (one-time disk read at init)
            char_config = load_character_config(p['config'])
            for v in char_config.get('name_variants', []):
                self._label_map[v.lower()] = p['label']

        # Build regex from known labels: \[(kelly|kelly bailey|kel|...)\]:
        # This matches multi-word names and avoids matching arbitrary text.
        label_alts = '|'.join(re.escape(k) for k in self._label_map.keys())
        self.LABEL_PATTERN = re.compile(
            r'\[(' + label_alts + r')\]:', re.IGNORECASE
        )

    def feed(self, raw_token):
        """Accept a raw token from the model. Returns list of structured events."""
        self.buffer += raw_token
        events = []

        # ── Check for <<YIELD>> token (might be split across chunks) ──
        if self.YIELD_TOKEN in self.buffer:
            # Yield complete — emit everything before it, then yield event
            before = self.buffer.split(self.YIELD_TOKEN)[0]
            if before and self.current_speaker:
                events.append({"speaker": self.current_speaker, "delta": before})
            events.append({"event": "yield"})
            self.buffer = self.buffer.split(self.YIELD_TOKEN, 1)[1]
            self.current_speaker = None
            return events

        # Check if buffer MIGHT contain a partial <<YIELD>> (prefix match)
        if self._has_partial_marker(self.buffer, self.YIELD_TOKEN):
            # Hold back only the unsafe portion, emit safe text first
            # (same split logic as partial labels — don't stall the whole buffer)
            safe, unsafe = self._split_safe_unsafe_yield(self.buffer)
            if safe and self.current_speaker:
                events.append({"speaker": self.current_speaker, "delta": safe})
                self.buffer = unsafe
```

```
            return events

        # ── Check for [Name]: label (might be split across chunks) ──
        label_match = self.LABEL_PATTERN.search(self.buffer)
        if label_match:
            name = label_match.group(1)
            speaker = self._resolve_label_to_speaker(name)
            if speaker:
                # Emit any text before the label (belongs to previous speaker)
                before_label = self.buffer[:label_match.start()]
                if before_label.strip() and self.current_speaker:
                    events.append({"speaker": self.current_speaker, "delta": before_label})

                # Switch speaker
                self.current_speaker = speaker
                after_label = self.buffer[label_match.end():]
                self.buffer = after_label

                if after_label:
                    events.append({"speaker": speaker, "delta": after_label})
                    self.buffer = ""
                return events

        # Check if buffer MIGHT contain a partial [Name]: label
        if self._has_partial_label(self.buffer):
            safe, unsafe = self._split_safe_unsafe(self.buffer)
            if safe and self.current_speaker:
                events.append({"speaker": self.current_speaker, "delta": safe})
                self.buffer = unsafe
            return events

        # ── No structural markers — emit as normal text for current speaker ──
        # current_speaker is never None here (initialized to default_speaker)
        if self.current_speaker and self.buffer:
            events.append({"speaker": self.current_speaker, "delta": self.buffer})
            self.buffer = ""

        return events

    def flush(self):
        """Emit any remaining buffered text. Call when stream ends."""
        events = []
        if self.buffer.strip() and self.current_speaker:
            events.append({"speaker": self.current_speaker, "delta": self.buffer})
        self.buffer = ""
        return events

    def _resolve_label_to_speaker(self, name):
        """Map a [Name]: label to a participant label. O(1) dict lookup, no disk."""
        return self._label_map.get(name.lower())

    def _has_partial_marker(self, text, marker):
        """Check if text ends with a prefix of marker (partial match)."""
        for i in range(1, len(marker)):
            if text.endswith(marker[:i]):
                return True
        return False

    def _has_partial_label(self, text):
        """Check if text might contain a partial [Name]: label at the end."""
        tail = text[-30:] if len(text) > 30 else text
        if '[' in tail:
            last_open = tail.rfind('[')
            after_open = tail[last_open:]
            if ']:' not in after_open:
                return True
        return False

    def _split_safe_unsafe(self, text):
        """Split text into safe-to-emit and hold-back portions (for labels)."""
        idx = text.rfind('[')
        if idx == -1:
            return text, ""
```

```
        remainder = text[idx:]
        if ']:' in remainder:
            return text, ""
        return text[:idx], text[idx:]

    def _split_safe_unsafe_yield(self, text):
        """Split text for partial <<YIELD>> markers. Emits safe text before '<'."""
        idx = text.rfind('<')
        if idx == -1:
            return text, ""
        # Check if this '<' could be start of <<YIELD>>
        remainder = text[idx:]
        if self.YIELD_TOKEN.startswith(remainder):
            return text[:idx], remainder
        return text, ""
```

Key fixes in TokenBufferQueue:

1. **`default_speaker` constructor param (Critical #1):** Single-character streams (the vast majority of turns) don't emit `[Name]:` labels. Without a default speaker, `current_speaker` is `None` and all text is silently dropped. Now `current_speaker` is initialized to `default_speaker` — the addressed character — so labelless streams work immediately.
2. **LABEL_PATTERN built from known labels (Critical #2):** The old `r'\[([A-Za-z]+)\]:'` couldn't match `[Kelly Bailey]:` (no `\s`). The new pattern is built from the actual label set at init time: `r'\[(kelly|kelly bailey|kel|kels|...)\]:'`. This matches multi-word display names, rejects unknown labels, and is case-insensitive.
3. **Label variants cached in `__init__` (Minor):** `_resolve_label_to_speaker` was calling `load_character_config()` on every label event during streaming — a disk read per token. Now all variants are pre-loaded into `_label_map` at construction time. `_resolve_label_to_speaker` is an O(1) dict lookup.
4. **Partial `<<YIELD>>` doesn't stall buffer (Minor):** The old code held back the entire buffer when a partial `<<YIELD>>` was detected. Now `_split_safe_unsafe_yield` emits safe text before the potential `<`, only holding back the unsafe prefix.
5. **Em-dash claim removed (Minor):** The docstring no longer claims em-dash parsing. Em-dashes are part of the interrupt event's `truncated_text`, not a stream structural marker.
   yield format_sse(event)

   yield format_sse({"event": "turn_complete"})

```
**Key principle:** The frontend's `handleStreamDelta()` function receives events with `speaker` already
resolved. It never parses `[Name]:` labels — it just checks `data.speaker` and routes to the correct
bubble. All the messy token-boundary logic lives in the backend where it belongs.

### 8.7 RAG Payload Formatting for Trio Mode

**The Problem:** Lagoon's existing RAG system chunks conversation into turn-pairs, embeds them with all-
MiniLM-L6-v2, and retrieves relevant chunks to inject into the context budget. In trio mode, those chunks
contain `[Name]:` labels — e.g., `[Nathan]: I hate community service.`

When retrieved, these chunks are injected as raw text. The model can't distinguish between:
- Active narrative happening right now in the conversation
- Recalled memory from earlier in the scene
- Recalled memory from a completely different scene/chapter

This causes temporal confusion — Kelly might read a RAG chunk and think Nathan just said "I hate community
service" moments ago, when it actually happened 3 chapters ago.

**The Solution:** Wrap all RAG retrieval results in a structural header that explicitly marks them as
recalled memory, not active narrative. The model is instructed (via combo prompt rules) to treat content
within these headers as background knowledge, not as something currently being said.

```python
def format_rag_for_trio(retrieved_chunks, scene_context=None):
    """Format RAG retrieval results with structural headers for trio mode.

    Wraps retrieved dialogue in clear memory markers so the model
    doesn't confuse recalled memory with active narrative.
    """
    if not retrieved_chunks:
        return ""

    header = "--- RETRIEVED SCENE MEMORY (Background Context — Not Active Dialogue) ---\n"
    if scene_context:
        header += f"Scene: {scene_context}\n"
    header += "The following are recalled fragments from earlier in the story. "
    header += "Treat these as background knowledge, NOT as something currently being said.\n\n"

    body = ""
    for i, chunk in enumerate(retrieved_chunks, 1):
        # Each chunk already has [Name]: labels from the conversation
        # Add a fragment number for reference
        body += f"[Fragment {i}]\n{chunk['text']}\n\n"

    footer = "--- END RETRIEVED SCENE MEMORY ---\n"

    return header + body + footer
```

Combo prompt addition (in RULES_TEMPLATE):

```
MEMORY HANDLING:
- Text between "--- RETRIEVED SCENE MEMORY ---" markers is background context.
- This is recalled memory, NOT active dialogue. Do not react to it as if
  someone just said it. Use it to inform your responses, but do not quote
  or directly reference it unless the conversation naturally calls for it.
- Active dialogue appears ONLY in the conversation history below, marked
  with [Name]: labels. That is what is happening NOW.
```

Per-character RAG awareness:

In trio mode, RAG chunks may contain dialogue from a character who isn't the current speaker. The `character_aware` flag on lorebook entries (already in Lagoon) should be tracked per-character, not globally. When Character A generates, only inject lore/RAG that Character A is aware of. If a secret was revealed only to Kelly in a previous scene, Nathan's RAG retrieval should not include that fragment.
```

```
def get_trio_rag_for_character(chat_id, character_label, query, ...):
    """Retrieve RAG chunks, filtered by what this character is aware of."""
    all_chunks = rag_retrieve(chat_id, query, ...)

    # Filter: only include chunks this character is aware of
    aware_chunks = []
    for chunk in all_chunks:
        # Check character_aware metadata on the chunk
        aware_of = chunk.get('metadata', {}).get('character_aware', {})
        if aware_of.get(character_label, True):  # Default: aware unless explicitly hidden
            aware_chunks.append(chunk)

    return format_rag_for_trio(aware_chunks)
```

---

9. Frontend Implementation

9.1 TrioManager.js

Extends DualModelManager pattern:

```
export const trioManager = {
    state: {
        participants: [],
        lastSpeaker: null,
        lastAddressee: null,
        activeThread: null,
        interruptCount: 0,
        isStreaming: false,
        currentSpeaker: null,
    },

    init(participants, dynamic) { ... },
    handleSendMessage(text) { ... },
    handleSSEEvent(event) { ... },
    handleInterrupt(event) { ... },
    renderParticipantStrip() { ... },
    letThemTalk() { ... },
    exitTrioMode() { ... },
};
```

9.2 Message Rendering

`messages.js` `addMessageToUI()` extended to handle speaker:

```
function addMessageToUI(role, content, options = {}) {
    const speaker = options.speaker || (role === 'user' ? 'user' : 'assistant');
    const participant = trioManager.getParticipant(speaker);

    const bubble = document.createElement('div');
    bubble.className = `message-group speaker-${speaker}`;
    bubble.style.borderLeftColor = participant?.color || 'var(--border)';

    if (options.interrupted) {
        bubble.classList.add('interrupted');
    }
    if (options.isInterruption) {
        bubble.classList.add('interruption');
    }

    // Avatar + name on EVERY message (not just first in group)
    const header = document.createElement('div');
    header.className = 'message-header';
    header.innerHTML = `
        <img src="${participant?.avatar || ''}" class="message-avatar">
        <span class="message-name" style="color: ${participant?.color}">
            ${participant?.display_name || 'Assistant'}
        </span>
    `;
    bubble.appendChild(header);

    // Content
    const body = document.createElement('div');
    body.className = 'message-content';
    body.innerHTML = parseMarkdown(content);
    bubble.appendChild(body);

    chatMessages.appendChild(bubble);
}
```

9.3 Participant Strip

Top bar showing who's in the conversation:

```
┌──────────────────────────────────────────────────────────┐
│     Kelly Bailey       Nathan Young       You           │
│  ──────────────      ──────────────                      │
│  (active)           (waiting)                            │
└──────────────────────────────────────────────────────────┘
```

- Active speaker gets a **glowing border** during streaming
- Clicking a participant avatar toggles @mention insertion in the input
- "Let them talk" button appears at the right end of the strip

9.4 Streaming Display

During streaming, SSE events route tokens to the correct bubble. **Critical:** the frontend receives pre-parsed structured events from the backend Token Buffer Queue (§8.6). The `speaker` field is already resolved — the frontend NEVER parses `[Name]:` labels from raw tokens. It just checks `data.speaker` and routes to the correct bubble:

```
function handleStreamDelta(data) {
    const { speaker, delta } = data;

    // speaker is pre-resolved by backend TokenBufferQueue
    // No string parsing of [Name]: labels on the frontend
    if (speaker !== currentStreamingSpeaker) {
        // Speaker changed — create new bubble
        finishCurrentBubble();
        startNewBubble(speaker);
    }

    appendToCurrentBubble(delta);
    autoScroll();
}

function handleInterruptEvent(data) {
    const { by, truncated_text } = data;

    // Finalize current speaker's bubble with em-dash
    appendToCurrentBubble('—');
    currentBubble.classList.add('interrupted');

    // Start interrupting speaker's bubble
    startNewBubble(by);
    appendToCurrentBubble('—');  // Leading em-dash
    currentBubble.classList.add('interruption');
}
```

---

10. Edge Cases & Error Handling

Scenario	Handling
Both characters have the same name	Disallow at config time; validate uniqueness
One character's API call fails	Stream error for that character; other character's response (if any) is preserved
Interrupt monitor fires but B's call fails	A's truncated message stays; system message: "Nathan was about to interrupt but faltered."
Model generates [You] lines	Strip them silently; log warning; do not display
Model generates both characters in a single-character call	Parse on [Name]: labels; if multiple speakers detected, split into separate messages
Model doesn't emit <> in let-them-talk	Hard cap at N rounds; stream cut after N
User types during streaming	Existing abort behavior; all in-flight streams cancelled
Model bleeds voices (Kelly sounds like Nathan)	Log occurrence; combo prompt rules + Style Overseer post-generation validation
Passive reaction is too long (model ignores max_tokens)	Hard truncate at max_tokens; log occurrence; consider lowering temperature
Passive reaction repeats what the main response said	Acceptable — reactions often echo; if verbatim, skip rendering (dedup check)
Emotional state update returns invalid JSON	Fall back to previous state; log warning; do not block the turn
Emotional state drifts unrealistically (guarded → euphoric in one turn)	Clamp only `energy` (ordinal: low/med/high, max one step per exchange). Categorical fields (`current` , `toward_other` , `toward_user`) are NOT clamped — they have no ordering. Instead, the emotional state update prompt instructs the model to shift gradually, and invalid jumps are logged for review.
Passive reaction fires every turn (feels spammy)	reaction_probability is an upper bound; model can return <>; energy modulation reduces frequency when energy is low
Both passive reaction and interrupt fire in same turn	Interrupt takes priority; passive reaction is skipped if an interrupt already occurred
Worst-case call count per user message exceeds budget	Per-turn call budget: max 6 LLM calls (1 main + 3 interrupts + 1 passive + 1 emotional). If an interrupt cycle occurs (≥2 interrupts), skip passive reaction AND emotional state update for that turn — the interrupt cycle already provided enough dynamism. Degraded mode flag: `degraded_mode: true` in trio_state.
Interrupt truncation point is mid-word or mid-markdown	Truncate at the last sentence/clause boundary before the trigger position, then append em-dash. Use `text[:pos].rpartition('. ')` or `text[:pos].rpartition(', ')` to find the boundary. "and it were all ju—" → "and it were all just—" (cleaner break).
User types `[Nathan]: I secretly love Kelly`	Sanitize user input: strip bracket-label patterns before adding to history. Regex `r'^\s*\[[A-Za-z\s]+\]:\s*'` is removed from user messages. The forged label is stripped, not preserved. Message becomes `[You]: I secretly love Kelly`.
Context window overflow	Existing summarization applies; combo prompt is counted as system message
One character has much longer responses	Patience threshold handles naturally; other character interrupts
User addresses a character who hasn't spoken yet	First message defaults to participants[0] if no last_speaker
Name detection matches a substring inside another word	Word-boundary regex prevents this; prefix fallback requires min 3 chars
User types a name that's a prefix of both characters (e.g. "Na" for Nathan/Nate)	Prefix fallback requires min 3 chars; "Na" (len 2) won't match. "Nat" matches Nathan only

Scenario	Handling
Both characters' names appear in the message	First-mentioned name is the addressee (vocative position); other is a referenced third party
User types "Kel..." with trailing punctuation	Word-boundary regex matches "Kel" because `...` is not a word character. Routes to Kelly
User types "Kellyyy" (drawn out)	Word-boundary fails, prefix fallback catches it: "kellyyy" starts with "kelly" (len >= 3)
Consecutive-turn counter gets stuck high	Reset on any explicit name detection, "both" keyword, or let-them-talk exchange
Content inference matches both characters equally	Fall through to tier 4 (last-speaker with alternation)
Name detection of non-existent character	No match in name_variants → falls through to tier 2
Interrupt keywords appear in user's message (not character's)	Monitor only watches character A's stream, not user messages
Token boundary splits a [Name]: label (e.g. `[K` then `elly]:`)	Backend TokenBufferQueue holds partial labels until complete, then emits structured SSE. Frontend never sees raw tokens
Token boundary splits <> (e.g. `<<YIE` then `LD>>`)	TokenBufferQueue checks for partial marker prefixes, holds back until complete
RAG retrieval injects past dialogue without context	All RAG chunks wrapped in `--- RETRIEVED SCENE MEMORY ---` headers; combo prompt instructs model to treat as background, not active dialogue
RAG chunk contains a secret one character doesn't know	Per-character `character_aware` metadata filters RAG per speaker; Nathan won't see chunks Kelly-only secrets
Model confuses recalled memory with active narrative	Structural headers + combo prompt rules explicitly distinguish the two

11. Build Phases

Phase 1: Foundation (Backend)

- [ ] Combo prompt builder function
- [ ] Message `speaker` field in data model
- [ ] `stream_chat` trio branch (sequential calls, no interrupts yet)
- [ ] SSE speaker tagging
- [ ] Per-character lore/RAG injection loop
- [ ] TokenBufferQueue class (buffer raw tokens, emit structured SSE)
- [ ] RAG payload formatting with `--- RETRIEVED SCENE MEMORY ---` headers
- [ ] Per-character RAG filtering via `character_aware` metadata

Phase 2: Addressing (Backend + Frontend)

- [ ] Name detection: word-boundary regex matcher (`_detect_names`)
- [ ] Name detection: prefix fallback matcher (`_detect_names_prefix`)
- [ ] "Both" keyword detection
- [ ] Content inference (addressing_keywords per character)

- [ ] Last-speaker default with forced alternation (consecutive_default_turns tracking)
- [ ] Counter reset logic (name/both resets, content inference doesn't)
- [ ] Trio state tracking (last_speaker, consecutive_default_turns, active_thread)
- [ ] Frontend: participant strip UI
- [ ] Frontend: per-speaker message rendering (colors, avatars, names)
- [ ] Character config: name_variants, nicknames, addressing_keywords fields
- [ ] Ambient responses toggle (ambient_responses: false default)

Phase 3: Interruptions (Backend + Frontend)

- [ ] Interrupt monitor class
- [ ] Live interrupt SSE protocol
- [ ] Frontend: interrupt visualization (em-dash, dashed borders)
- [ ] Resume call logic
- [ ] Guard rails (max interrupts, cooldown)

Phase 4: Let Them Talk (Backend + Frontend)

- [ ] Single-call dual-response mode
- [ ] [Name]: label parsing (in TokenBufferQueue, NOT frontend)
- [ ] <> token handling (in TokenBufferQueue, detects split tokens)
- [ ] Partial marker detection (hold back tokens that might be partial labels/yields)
- [ ] "Let them talk" button UI
- [ ] Round limiting

Phase 5: Presence & Emotional Systems

- [ ] Passive presence system (§6.6): reaction_probability roll, <> token
- [ ] Passive reaction API call (lightweight, max 80 tokens)
- [ ] Emotional state tracking (§6.7): per-character mood tracker
- [ ] Emotional state update call (post-exchange, JSON response)
- [ ] Emotional state injection into combo prompt PRESENCE section
- [ ] Energy-based reaction probability modulation
- [ ] Frontend: passive reaction styling (muted, smaller, no header)
- [ ] Frontend: emotional state indicators (subtle, in participant strip)
- [ ] "Both hear everything" combo prompt rule (§6.8)

Phase 6: Polish

- [ ] Trio chat save/load (participants[], trio_state, emotional_state all persisted). Reloading a chat mid-arc must preserve last_speaker, consecutive counters, and emotional state — otherwise Nathan snaps back to "performing / high energy" and §6.7's arc is defeated.
- [ ] Relationship dynamic editor UI
- [ ] Addressing keywords editor in character config
- [ ] Interrupt keywords/patience editor in participant config
- [ ] Visual theming (zigzag connectors, glow effects)
- [ ] Mobile responsive layout
- [ ] (v2+) LLM router fallback for Tier 4a (gated behind use_llm_router flag)
- [ ] (v2+) Motivation-based interrupts (§6.9, gated behind motivation_based_interrupts flag)

Phase 7: Testing

- [ ] Unit tests: combo prompt builder, addressing resolver, interrupt monitor

- [ ] Integration tests: full trio turn cycle, interrupt cycle, let-them-talk
- [ ] Voice bleed detection: automated check for cross-character vocabulary
- [ ] Load testing: parallel interrupt monitoring performance

---

12. Testing Strategy

12.1 Unit Tests

```
test_combo_prompt_builder():
    - Two characters → correct structure with all sections
    - Missing dynamic fields → graceful defaults
    - Character config missing fields → graceful defaults

test_name_detection_word_boundary():
    - "Kelly" → kelly (exact match)
    - "Kel" → kelly (nickname, word-boundary)
    - "Kel..." → kelly (trailing punctuation, \b matches after "kel")
    - "@Kelly" → kelly (@ stripped, then matched)
    - "excellent" → no match ("kel" inside "excellent" blocked by \b)
    - "Nate" → nathan (nickname)
    - "NY" → nathan (initials)

test_name_detection_prefix_fallback():
    - "Kellyyy" → kelly (prefix match, len >= 3)
    - "Nath" → nathan (prefix match)
    - "Ke" → no match (len < 3, too short)
    - "Na" → no match (len < 3)

test_name_detection_both_characters():
    - "Kelly, tell Nathan" → kelly (first-mentioned = addressee)
    - "Nathan, what does Kelly think" → nathan (first-mentioned)
    - "Kelly and Nathan" → kelly (first-mentioned, even with "and")

test_addressing_resolver_full():
    - "@Kelly" → kelly (tier 1, @ optional)
    - "Kel... you know that's not true" → kelly (tier 1, nickname + ellipsis)
    - "you two" → both (tier 2)
    - "reading his mind" → kelly (tier 3, keyword)
    - "how many times have you died" → nathan (tier 3, keyword)
    - "yeah?" → last_speaker (tier 4, no cues)
    - "@Unknown" → last_speaker (no match, fallback)

test_consecutive_alternation():
    - Kelly is last_speaker, consecutive=0, vague msg → kelly, consecutive=1
    - Kelly is last_speaker, consecutive=1, vague msg → kelly, consecutive=2
    - Kelly is last_speaker, consecutive=2, vague msg → NATHAN (forced switch)
    - After forced switch, both counters reset to 0
    - Name detection resets both counters to 0
    - "both" keyword resets both counters to 0
    - Content inference does NOT reset counters
    - After 5 keyword-matched Kelly turns, vague msg → kelly (consecutive still 0)

test_interrupt_monitor():
    - <<YIELD>> in stream → 'yield' trigger
    - Keyword in stream → 'keyword' trigger
    - Stream exceeds patience → 'patience' trigger
    - Max interrupts reached → None (no more interrupts)
    - Cooldown active → None (suppressed)

test_token_buffer_queue():
    - Complete [Kelly]: label in one chunk → emit with speaker=kelly
    - Label split across chunks: "[K" + "elly]:" → hold, then emit when complete
    - <<YIELD>> split: "<<YIE" + "LD>>" → hold, then emit yield event
    - Text before label: "blah [Kelly]:" → emit "blah" for prev speaker, then switch
    - No label in stream → emit as current_speaker text
    - Flush on stream end → emit remaining buffer
    - Partial label at end: "some text [Kel" → emit "some text ", hold "[Kel"
    - Unknown label: "[Unknown]:" → no speaker match, treat as text

test_rag_formatting():
    - Empty chunks → empty string
    - Single chunk → wrapped in RETRIEVED SCENE MEMORY headers
    - Multiple chunks → numbered fragments [Fragment 1], [Fragment 2]
    - character_aware filtering: chunk with kelly:false → excluded for kelly
    - character_aware filtering: chunk with kelly:true → included for kelly
    - Default (no metadata) → included (aware unless explicitly hidden)

test_llm_router_fallback():  # v2+ only
    - LLM returns 'kelly' → route to kelly
    - LLM returns 'nathan' → route to nathan
```

```
    - LLM returns 'both' → route to both
    - LLM returns 'last' → fall through to heuristic (last speaker)
    - LLM returns unparseable → fall through to heuristic
    - use_llm_router: false → never calls LLM, uses heuristic directly
    - Simple message ("yeah") → heuristic, no LLM call (not complex enough)
    - Emotional message after vulnerable Kelly turn → LLM router invoked

test_passive_presence():
    - reaction_probability: 0.0 → never reacts
    - reaction_probability: 1.0 → always rolls (model may still return <<SILENT>>)
    - <<SILENT>> in response → no reaction rendered
    - Energy 'low' → probability reduced by 0.15
    - Energy 'high' → probability increased by 0.15
    - passive_presence_enabled: false → never calls reaction API
    - Interrupt occurred this turn → passive reaction skipped
    - Both characters spoke (Tier 2) → passive reaction skipped
    - Reaction exceeds max_tokens → hard truncate

test_emotional_state():
    - Initial state loaded from config
    - After exchange → state updated via API call
    - Invalid JSON response → fall back to previous state
    - State injected into combo prompt PRESENCE section
    - Energy modulates passive reaction probability
    - State persists across turns (not reset each turn)
    - Clamp: max one step change per field per exchange
```

12.2 Integration Tests

```
test_full_trio_turn():
    - User sends message → addressing resolves → character responds → SSE streams → message rendered

test_interrupt_cycle():
    - Character A streams → keyword triggers B → B interrupts → A resumes → A yields → turn complete

test_let_them_talk():
    - Button clicked → single call → response parsed into N messages → all rendered correctly

test_voice_bleed():
    - Generate 10 exchanges → check Kelly's messages don't contain Irish slang
    - Check Nathan's messages don't contain northern English dialect
```

---

13. Open Questions

1. **Should "let them talk" mode allow the user to inject mid-exchange?** (e.g., user types during character banter → does it interrupt the flow?) — *Recommendation: yes, user input always takes priority.*
2. **Should interrupt keywords be regex or simple string match?** — *Recommendation: start with simple string match (case-insensitive), upgrade to regex if needed.*
3. **Should the addressing resolver use an LLM call for ambiguous cases?** — *Decision: heuristic for v1 (fast, free, handles 90%+ of cases). Optional LLM router as Tier 4a fallback for v2+ — a tiny 1B parameter model or all-MiniLM classifier that reads conversation history and classifies intent ('kelly', 'nathan', 'both', 'last'). Only invoked for emotionally complex ambiguous messages where the heuristic would misroute. Gated behind `use_llm_router: false` by default. Enable if users report misrouting in complex emotional scenes. See §5.5 for implementation.*
4. **How many participants maximum?** — *Recommendation: 2 characters + 1 user for v1. Architecture supports more, but interrupt logic and UI get complex.*
5. **Should characters have individual memory (RAG) or shared?** — *Decision: shared conversation RAG (one chat ID), per-character context RAG (each character's own context file). RAG chunks are filtered per-character via `character_aware` metadata — secrets revealed to one character don't leak to the other. All RAG injections wrapped in `--- RETRIEVED SCENE MEMORY ---` structural headers (§8.7).*

6. **Should the TokenBufferQueue live in the backend or frontend?** — *Decision: backend. The frontend must NEVER receive raw model tokens in trio mode. All structural marker parsing ([Name]: labels, <>, em-dashes) happens server-side in the TokenBufferQueue (§8.6). The frontend receives only structured SSE events with pre-resolved `speaker` fields. This prevents mid-token parsing failures when the model emits tokens as word fragments.*
7. **Should the combo prompt be cached between calls?** — *Decision: yes, with split caching. The combo prompt is split into two parts: (1) a static prefix (character definitions, relationship dynamics, behavioral rules — everything that doesn't change within a chat session) and (2) a dynamic suffix (emotional state, recent context, turn-specific instructions). Only the static prefix is cached. The dynamic suffix is appended fresh each turn. This also plays nicely with provider-side prefix caching (Venice/OpenAI cache the first N tokens of a prompt automatically). Cache key = hash of both character configs + relationship dynamic + rules version. Invalidated only when character configs change, not when emotional state updates.*
8. **One model or two?** — *Decision: ONE model (Option A — locked). Both characters are written by the same model via the combo prompt. Per-character model selection is NOT supported. This is a deliberate architectural choice: the combo prompt, TokenBufferQueue, let-them-talk single-call design, and em-dash interrupt protocol all require one model writing both characters. See §4.3 for full rationale. Voice bleed is mitigated by combo prompt rules and the Style Overseer.*
9. **How do we solve the "missing character" problem?** — *Decision: Passive Presence System (§6.6). After the addressed character responds, the non-speaking character gets a probability-based reaction roll. Nathan (60%) is more likely to react than Kelly (35%). The model can opt out with <>. This keeps both characters present in every scene, even when only one is speaking. Combined with emotional state tracking (§6.7) and the "both hear everything" combo prompt rule (§6.8), this transforms the experience from "talking to one character at a time" to "being in a room with two people."*

---

14. Appendix: Kelly & Nathan Example Session

A full example session demonstrating all features:

```
═══ PARTICIPANT STRIP ═══
   Kelly Bailey (active)      Nathan Young (waiting)      You
──────────────────────────────────────────────────────────

[You]: So Kelly, what's it actually like? Hearing thoughts?

    → Addressing: @Kelly (no @, but "hearing thoughts" → Kelly keyword)
    → Route to Kelly

[   Kelly]: It's not like turning on a tap, is it? It's just...
    always there. Every thought, every horrible little thing people
    think but don't say. You can't—

    → Interrupt monitor: Nathan's keyword trigger? No.
    → Patience threshold: Nathan's threshold = 150 chars. Stream = 187 chars.
    → PATIENCE TRIGGER FIRES.

[SSE: {"event": "interrupt", "by": "nathan"}]

    → Kelly's message truncated with —
    → Nathan's call begins

[   Nathan]: —can't what? Can't handle it? Please. I've DIED.
    Multiple times. You don't see me complaining about—well, okay,
    I complain about it constantly, but the point stands.

    → No interrupt from Kelly (her keywords don't match)
    → Resume available for Kelly

[SSE: {"event": "resume_available", "for": "kelly"}]

    → Kelly's resume call: "You were interrupted. Continue or yield."

[   Kelly]: I was going to say you can't switch it off. But go off,
    Nathan. Tell us more about how dying is the same as hearing
    every horrible thought people have about you.

    → <<YIELD>> not emitted. Kelly finishes naturally.
    → Turn complete.

[SSE: {"event": "turn_complete"}]

──────────────────────────────────────────────────────────

[You]: You're both idiots.

    → "both" keyword detected → Route to both (sequential)

[   Kelly]: Cheers.
    → Appended to history

[   Nathan]: I'm the idiot? She's the one who can't—okay, fair.
    → Sees Kelly's "Cheers" → responds to it
    → Appended to history

──────────────────────────────────────────────────────────

[User clicks "LET THEM TALK"]

    → Single API call, dual-response mode
    → Combo prompt: "Current turn: Let them talk. Exchange 2 rounds."

[   Kelly]: You always do this. Make everything about you.
[   Nathan]: Everything IS about me. I'm immortal. That's literally
    my superpower. Being the main character.
[   Kelly]: Your superpower is being annoying.
[   Nathan]: And yet you keep talking to me.
[   Kelly]: <<YIELD>>

    → Yield token detected. Exchange ends.
```

```
    → 5 messages parsed and rendered.
    → Control returns to user.
```

---

15. References

- Existing `DualModelManager.js` — pattern for multi-model management
- `routes/chat.py` `stream_chat()` — current single-character streaming implementation
- `services/anchors.py` — lore injection (to be looped per character)
- `services/rag.py` — conversation memory (shared, not per-character)
- `services/context_rag.py` — character context file RAG (per-character)
- `js/ui/messages.js` `addMessageToUI()` — message rendering (to be extended with speaker)
- `js/state.js` — chat state management (to be extended with trioState)

End of document.